JAVASCRIPT PROGRAMMING LAB (BCS-354)

B.Tech CSE 3rd SEMESTER

SUBMITTED TO:

MR. AMAN

SUBMITTED BY:

RIYA CHAUDHARY

Roll No: 2510303439

Section: C

SN.	PROGRAM NAME	PAGE	DATE	SIGNATURE
1.	Your First JavaScript Script	3		
2.	Operators, Type Conversion and a Calculator	7		
3.	Conditions and Loops	10		
4.	Writing Functions	13		
5.	Arrays, Array Methods and Nested Arrays	16		
6.	JSON Text	20		
7.	Constructor Functions and Classes	23		
8.	Inheritance, Methods and Composition	26		
9.	DOM, Validation, Storage and Quiz	30		
10.	Objects and Destructuring	37		
11.	Scope and Closures	40		
12.	Callbacks and Higher-Order Functions	43		
13.	Reduce, Some, Every and Map	46		
14.	Timers and Intervals	49		
15.	Promises and async/await with Local Data	52		

Assignment 1: Your First JavaScript Script

Assignment Aim / Objective: To run a script in the browser, declare values with const and let, and find the type of a value using typeof.

Task 1: Hello from the script

Problem Statement: Create an HTML page with a script tag in the body. Print Hello from the script with console.log. Open the page and check the Console (F12).

Source Code (index.html):

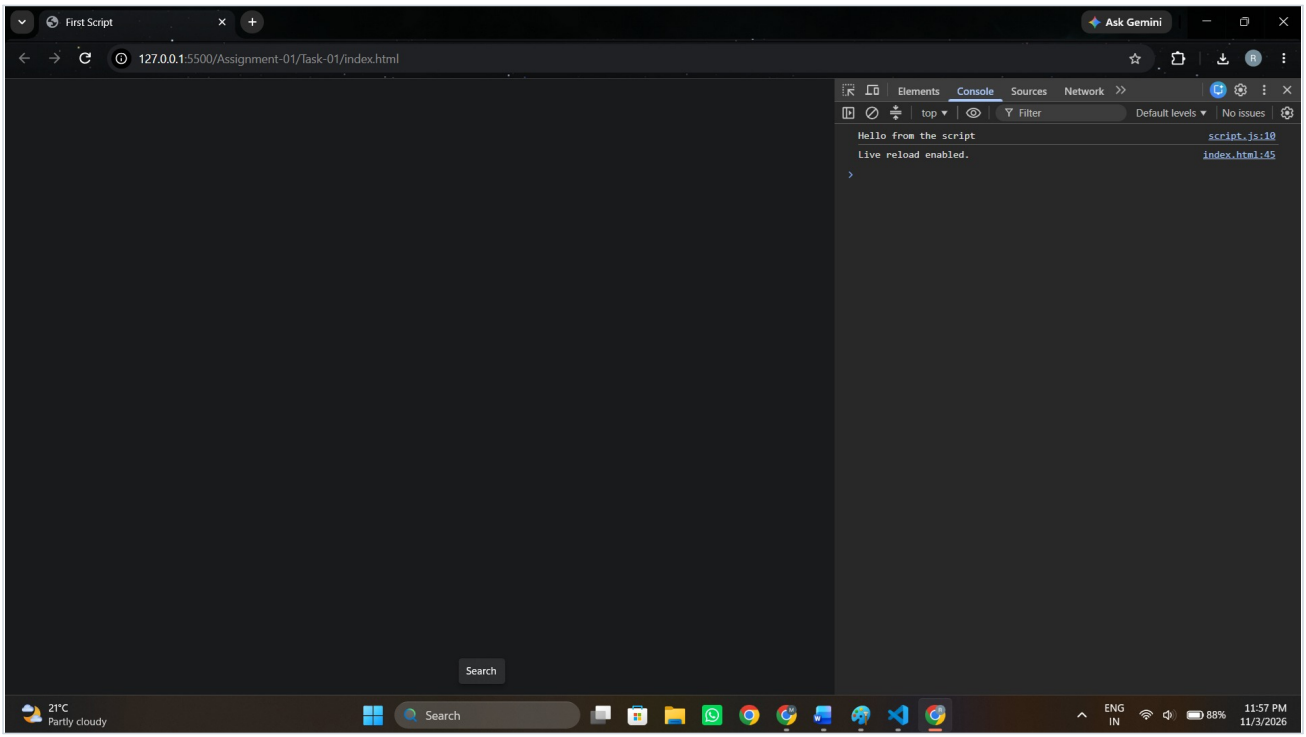
```
<!--
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 1, Task: 1
-->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>First Script</title>
</head>
<body>
<script src="script.js"></script>
</body>
</html>
```

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 1, Task: 1
 */

console.log("Hello from the script");
```

Execution Output Screenshot:



Task 2: const and let

Problem Statement: Declare const schoolName = "Sample College". Declare let year = 2026. Print both values on separate lines.

Source Code (script.js):

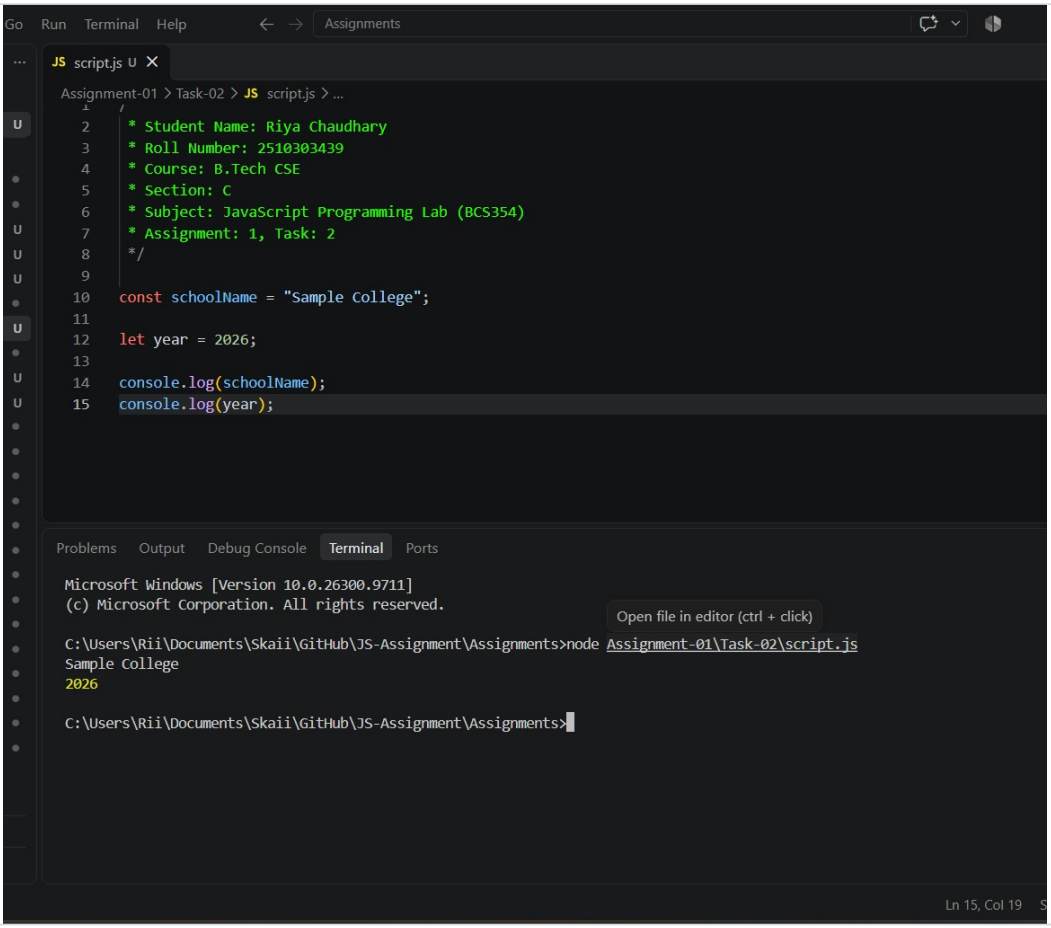
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 1, Task: 2
 */

const schoolName = "Sample College";

let year = 2026;

console.log(schoolName);
console.log(year);
```

Execution Output Screenshot:



Task 3: typeof

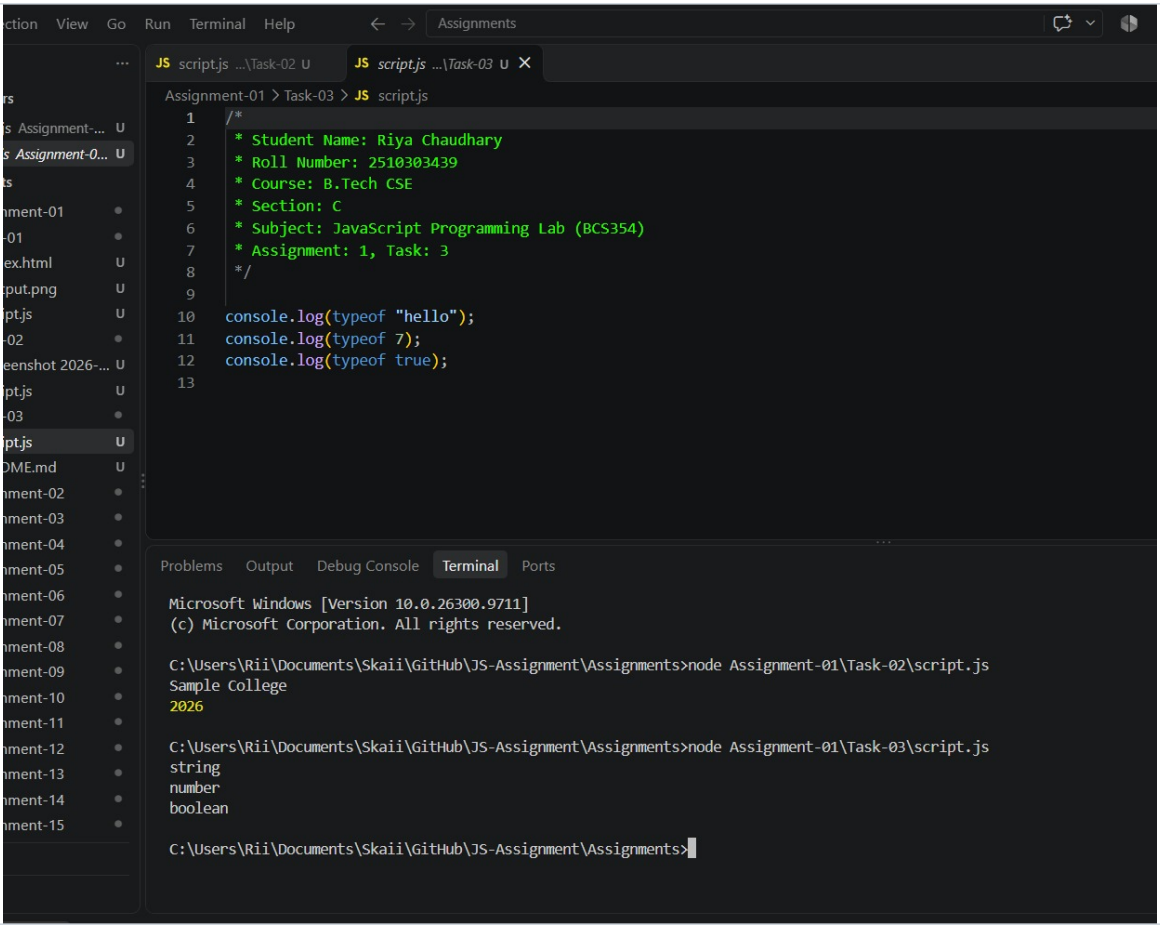
Problem Statement: Print typeof "hello". Print typeof 7. Print typeof true.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 1, Task: 3
 */

console.log(typeof "hello");
console.log(typeof 7);
console.log(typeof true);
```

Execution Output Screenshot:



Assignment 2: Operators, Type Conversion and a Calculator

Assignment Aim / Objective: To use arithmetic operators, convert between text and numbers, and use if and else if to choose an operation.

Task 1: Arithmetic operators

Problem Statement: Store 17 and 5 in variables a and b. Print a + b, a - b, a * b and a / b.

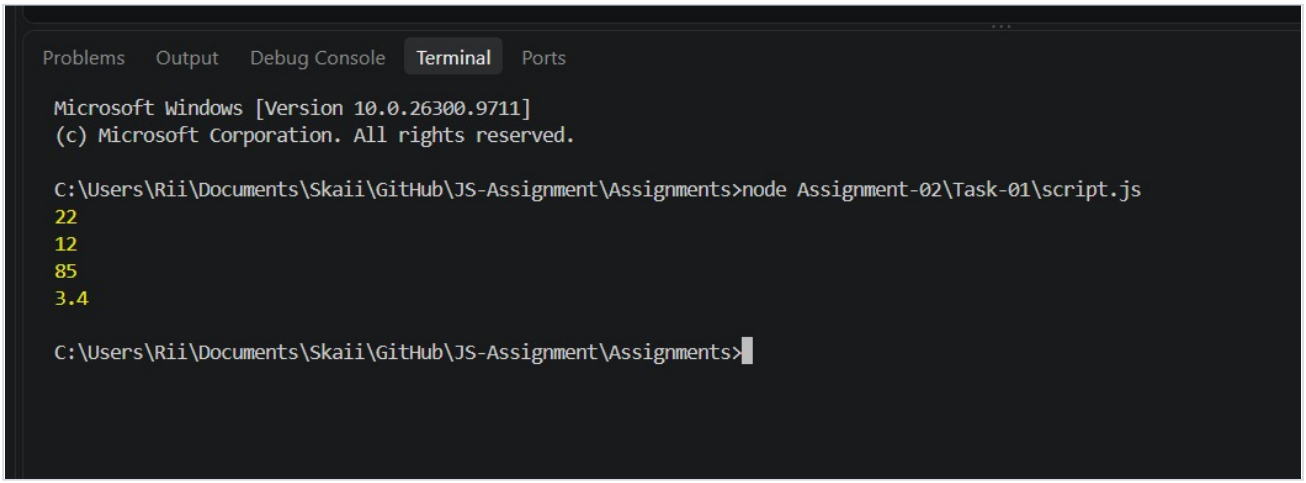
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 2, Task: 1
 */

let num1 = 17;
let num2 = 5;

console.log(num1 + num2);
console.log(num1 - num2);
console.log(num1 * num2);
console.log(num1 / num2);
```

Execution Output Screenshot:



Task 2: Type conversion

Problem Statement: Convert the text "12" to a number with Number() and add 5. Convert the number 7 to text with String() and join it with "3". Print both results.

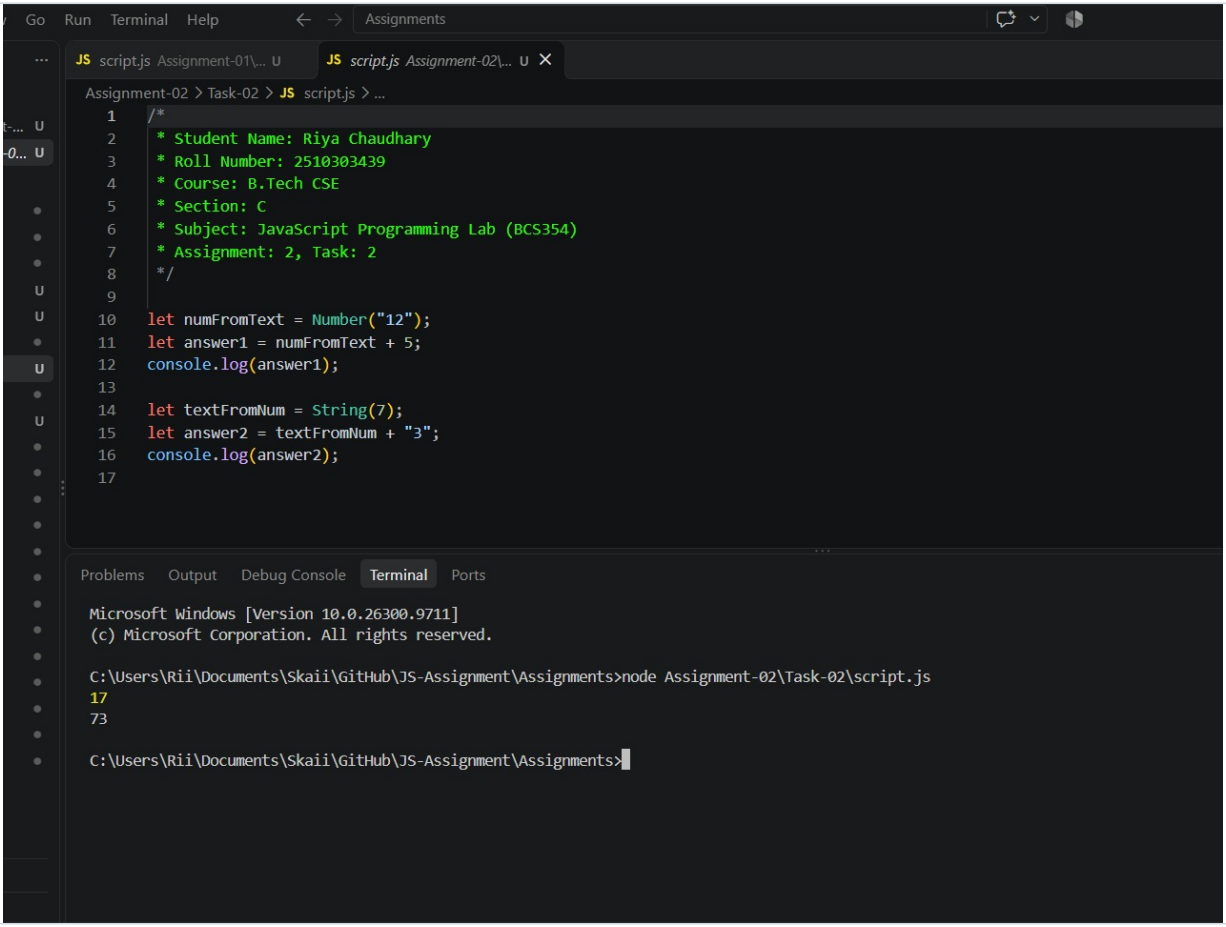
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 2, Task: 2
 */

let numFromText = Number("12");
let answer1 = numFromText + 5;
console.log(answer1);

let textFromNum = String(7);
let answer2 = textFromNum + "3";
console.log(answer2);
```

Execution Output Screenshot:



Task 3: Simple calculator

Problem Statement: Store 8 and 4 and the operator +. Use if and else if to print the sum for + or the difference for -. Change the operator to - and repeat the check.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 2, Task: 3
 */

let a = 8;
let b = 4;
let op = "+";

if (op == "+") {
  console.log(a + b);
} else if (op == "-") {
  console.log(a - b);
}

op = "-";

if (op == "+") {
  console.log(a + b);
} else if (op == "-") {
  console.log(a - b);
}
```

Execution Output Screenshot:



Assignment 3: Conditions and Loops

Assignment Aim / Objective: To choose between cases with if, else if and else, and to repeat work with a for loop and a while loop.

Task 1: Grade from marks

Problem Statement: Store the marks 72. If marks >= 75 print Grade A, else if marks >= 60 print Grade B, otherwise print Grade C.

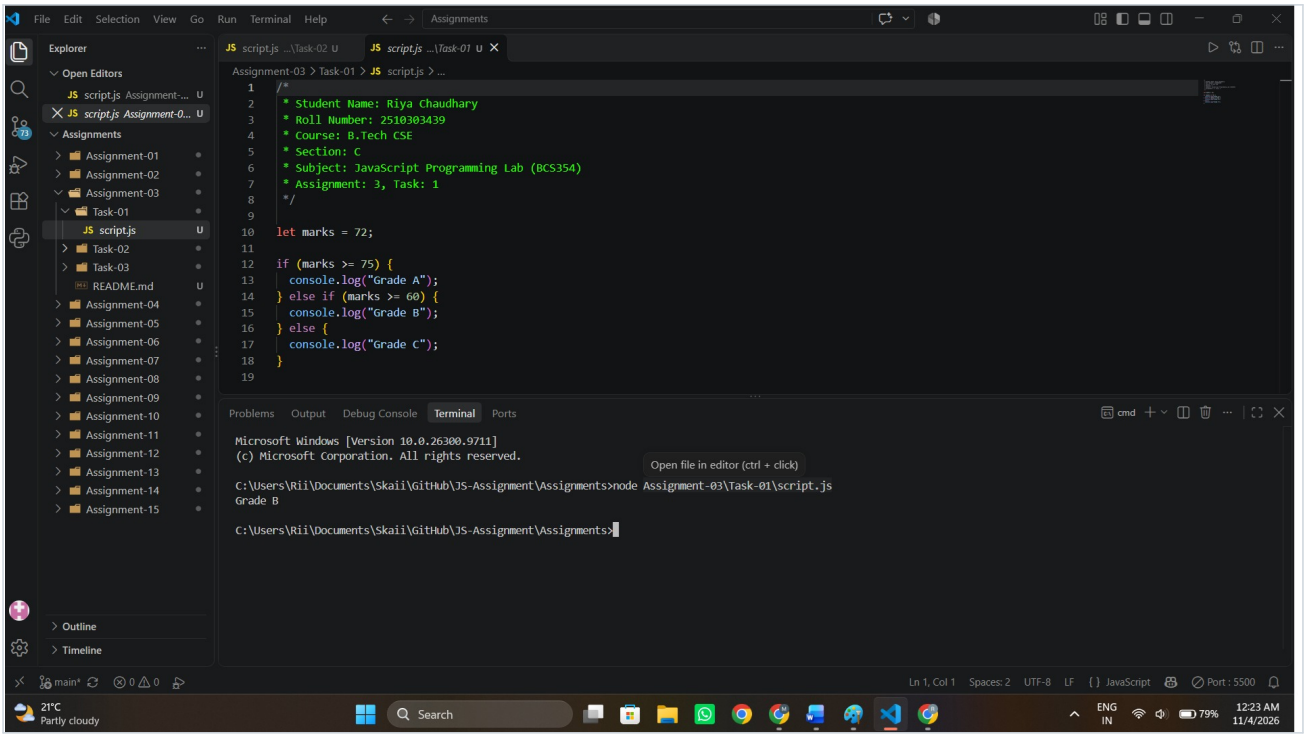
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 3, Task: 1
 */

let marks = 72;

if (marks >= 75) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

Execution Output Screenshot:



Task 2: for loop

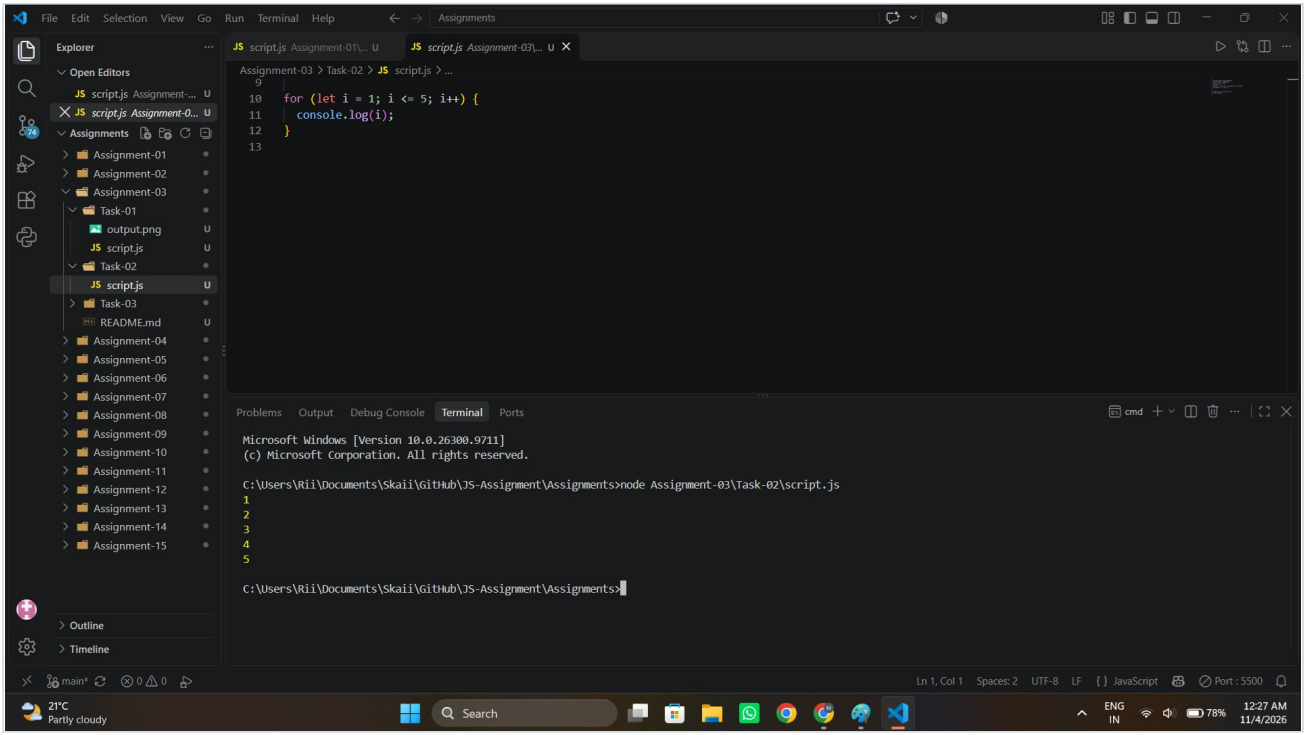
Problem Statement: Use a for loop with i from 1 to 5. Print i on each pass.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 3, Task: 2
 */

for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Execution Output Screenshot:



Task 3: while loop

Problem Statement: Set total = 0 and n = 1. While total is not more than 20, add n to total and increase n by 1. Print the total.

Source Code (script.js):

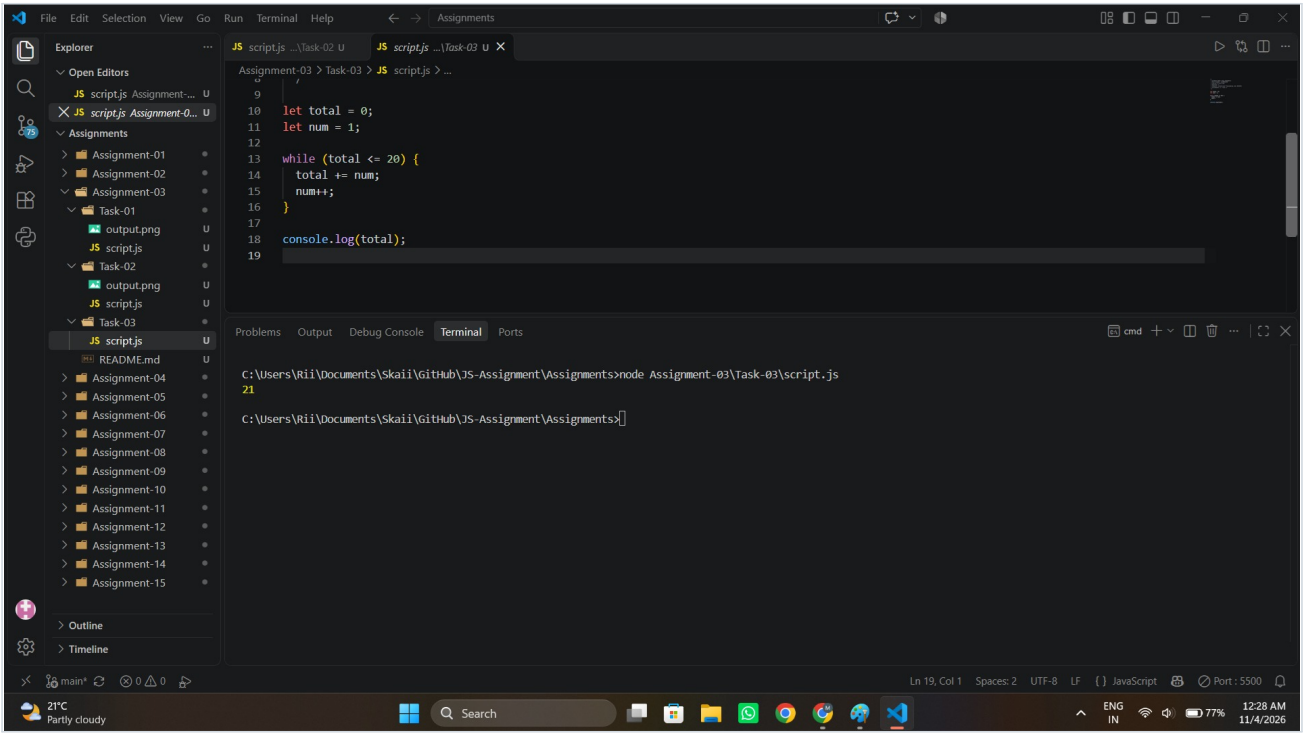
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 3, Task: 3
 */

let total = 0;
let num = 1;

while (total <= 20) {
  total += num;
  num++;
}

console.log(total);
```

Execution Output Screenshot:



Assignment 4: Writing Functions

Assignment Aim / Objective: To write a function with a return value, an arrow function, and a function with a default parameter.

Task 1: greet function

Problem Statement: Create function greet(name) that returns "Hello, " + name. Print greet("Asha").

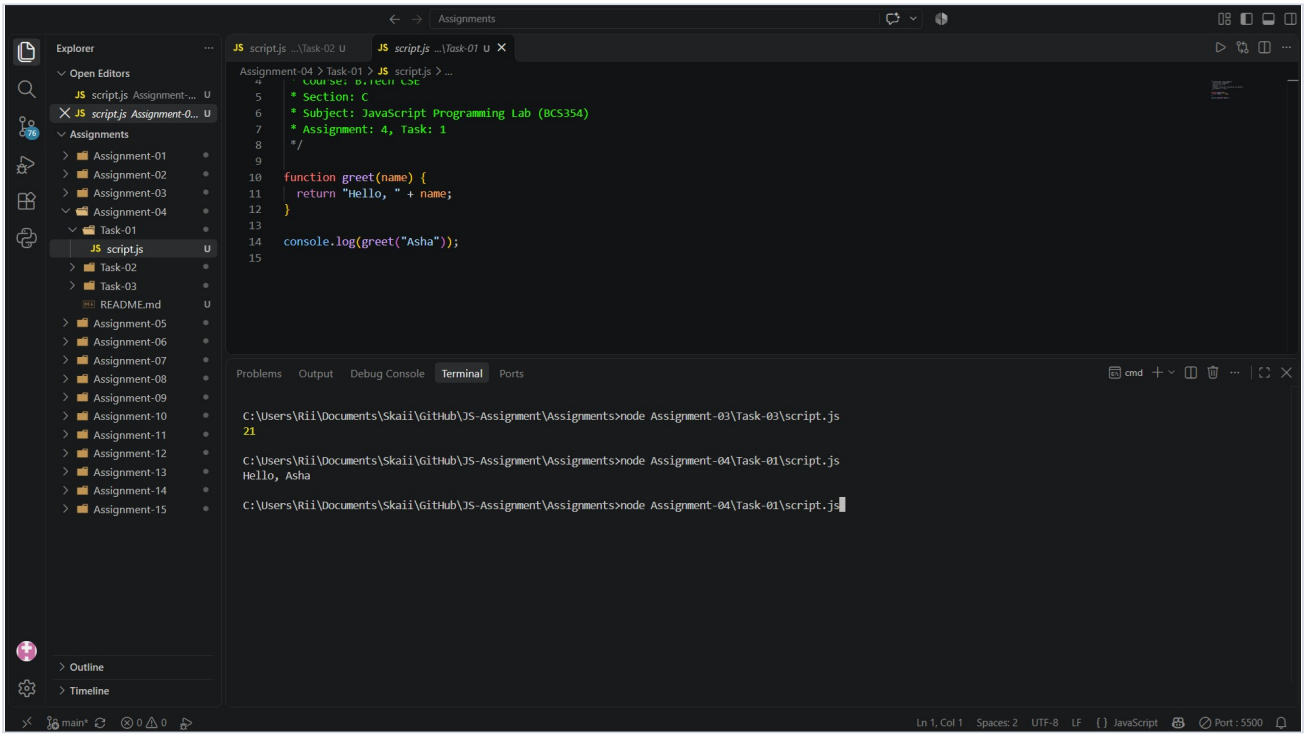
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 4, Task: 1
 */

function greet(name) {
  return "Hello, " + name;
}

console.log(greet("Asha"));
```

Execution Output Screenshot:



Task 2: Arrow function square

Problem Statement: Create the arrow function square that returns n * n. Print square(6).

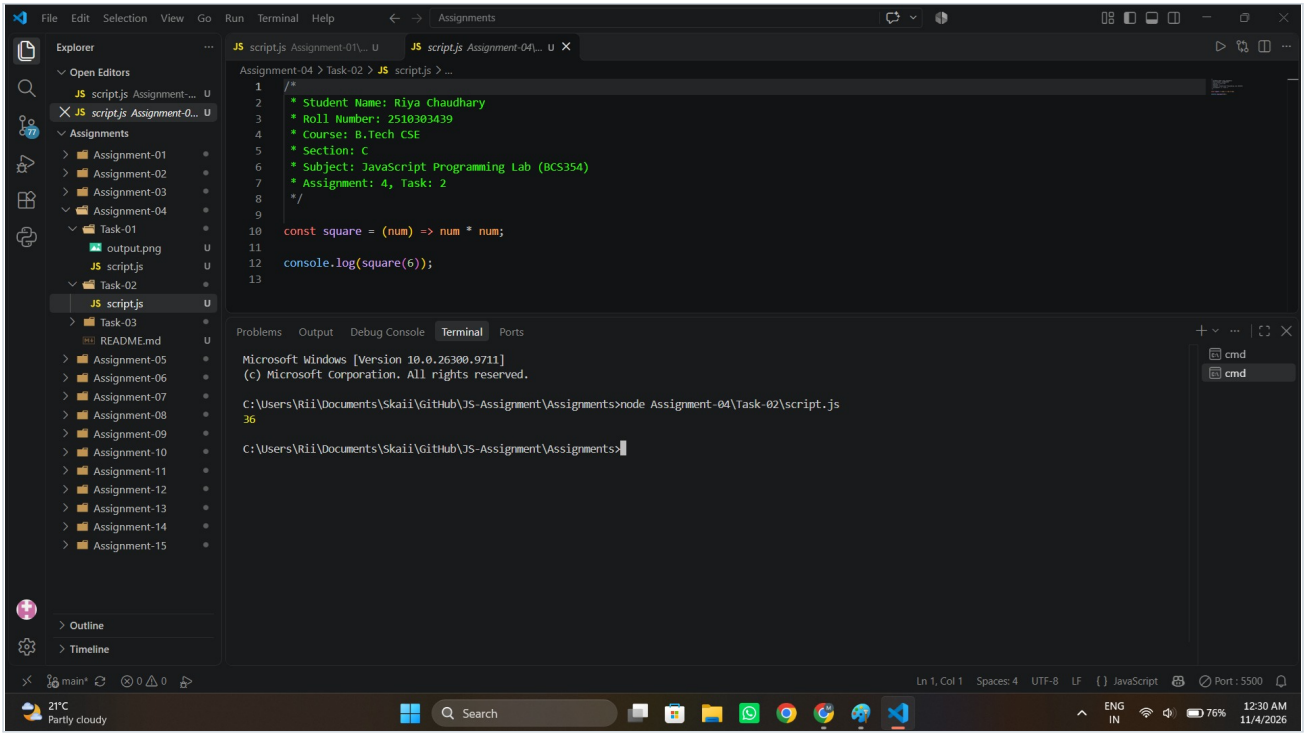
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 4, Task: 2
 */

const square = (num) => num * num;

console.log(square(6));
```

Execution Output Screenshot:



Task 3: Default parameter

Problem Statement: Create billWithTax(amount, taxRate = 0.18) returning amount + amount * taxRate. Print billWithTax(100).

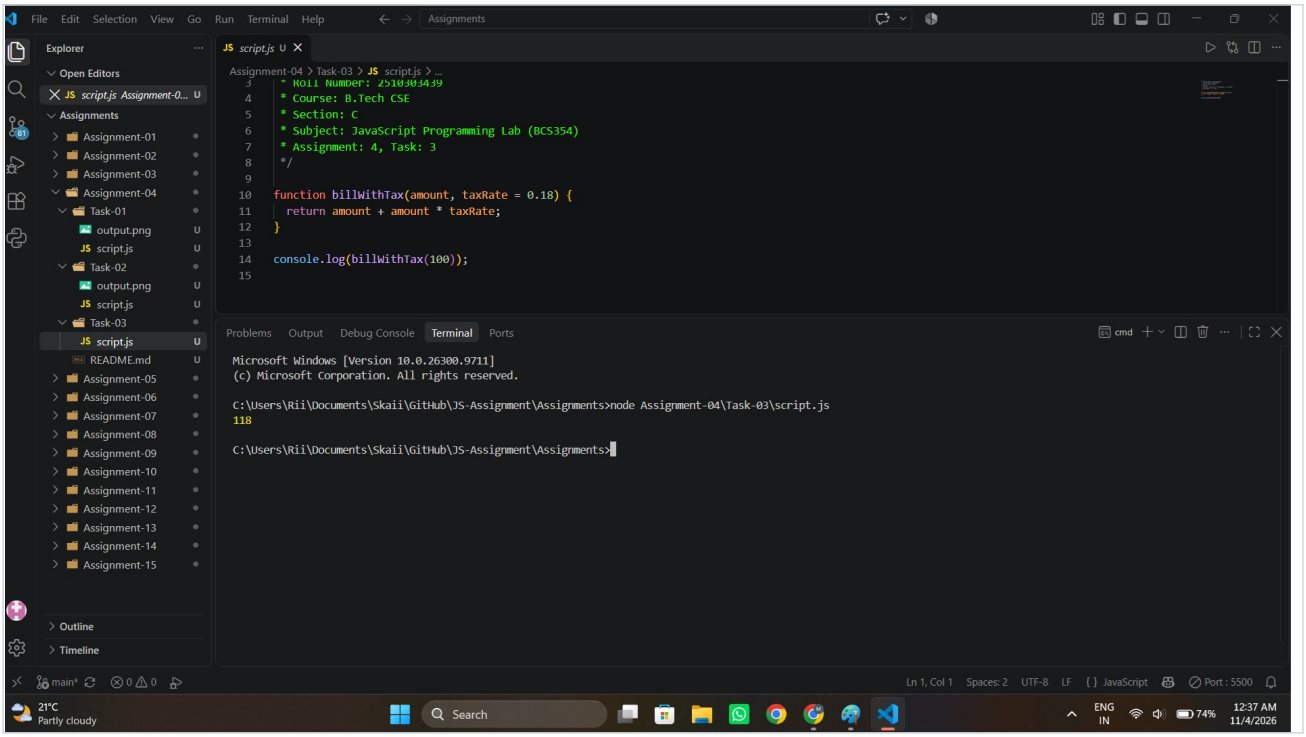
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 4, Task: 3
 */

function billWithTax(amount, taxRate = 0.18) {
  return amount + amount * taxRate;
}

console.log(billWithTax(100));
```

Execution Output Screenshot:



Assignment 5: Arrays, Array Methods and Nested Arrays

Assignment Aim / Objective: To create an array, read its items, use forEach and filter, and read a value from an array inside an array.

Task 1: Array length and first item

Problem Statement: Store 72, 85, 64 and 90 in the array marks. Print marks.length and marks[0].

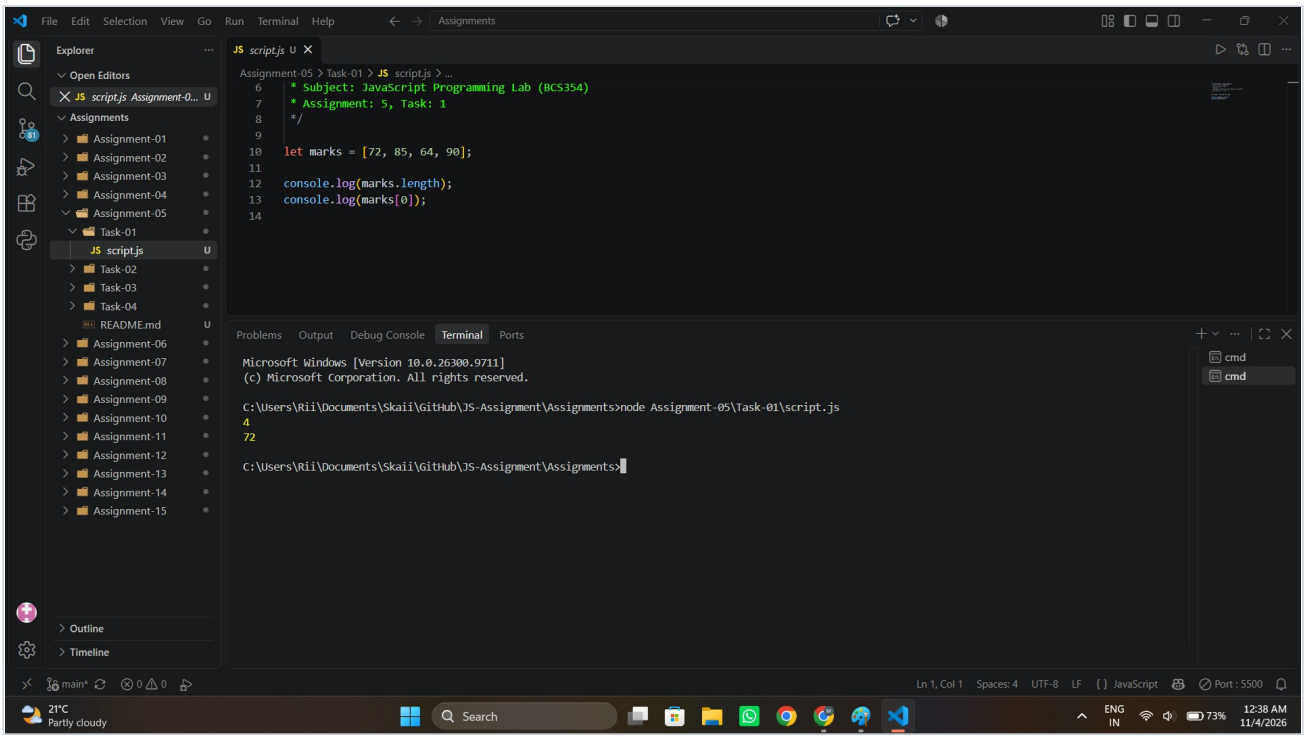
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 5, Task: 1
 */

let marks = [72, 85, 64, 90];

console.log(marks.length);
console.log(marks[0]);
```

Execution Output Screenshot:



Task 2: forEach

Problem Statement: Store Maths, Science and English in an array. Use forEach to print each subject.

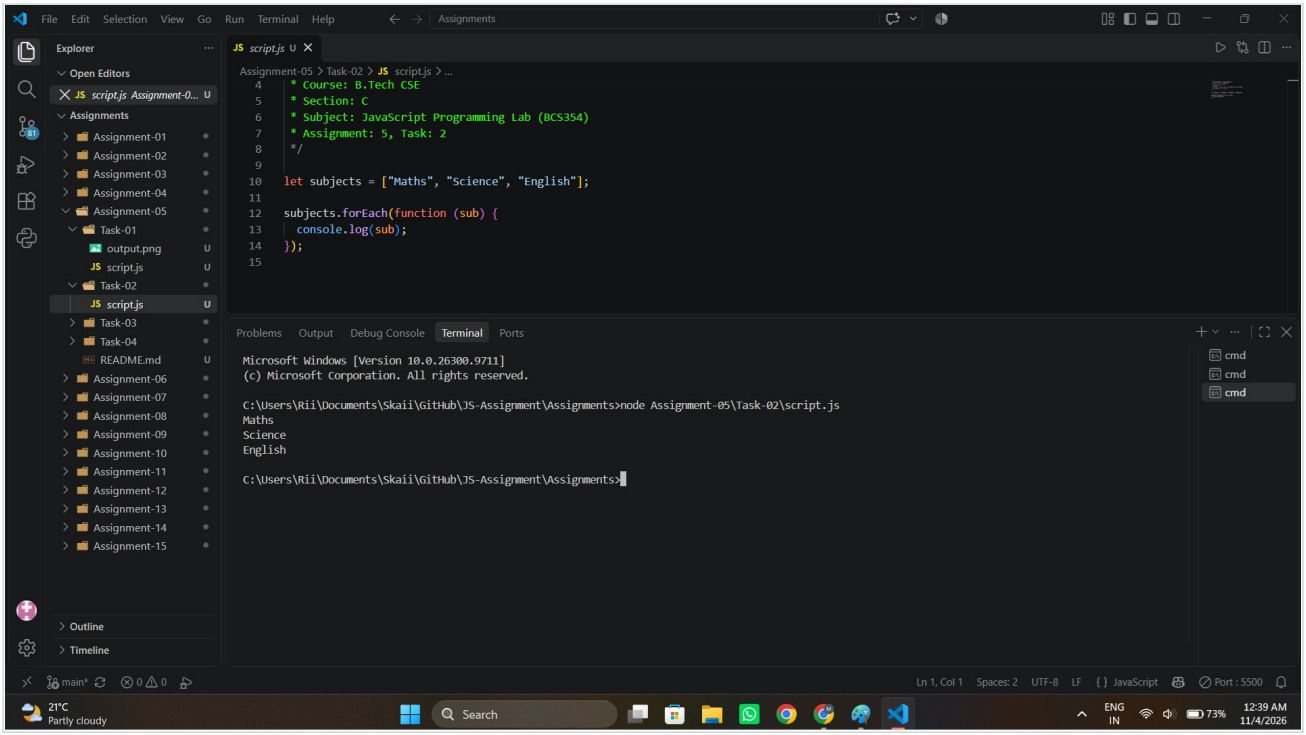
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 5, Task: 2
 */

let subjects = ["Maths", "Science", "English"];

subjects.forEach(function (sub) {
  console.log(sub);
});
```

Execution Output Screenshot:



Task 3: filter

Problem Statement: Store 72, 85, 45 and 90 in an array. Use filter to keep marks of 60 or more. Print the new array.

Source Code (script.js):

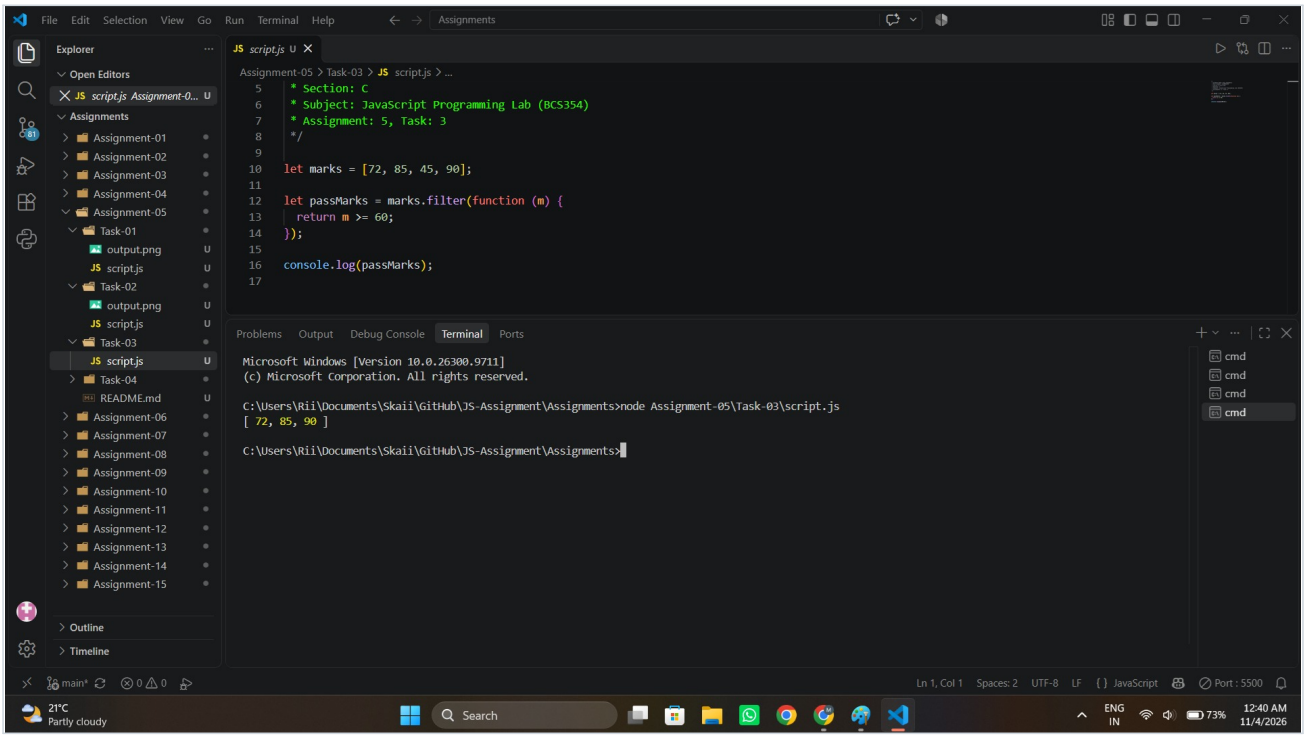
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 5, Task: 3
 */

let marks = [72, 85, 45, 90];

let passMarks = marks.filter(function (m) {
  return m >= 60;
});

console.log(passMarks);
```

Execution Output Screenshot:



Task 4: Nested array

Problem Statement: Store [1, 2, 3] and [4, 5, 6] inside the array grid. Print the number in row 2 and column 3, which is grid[1][2].

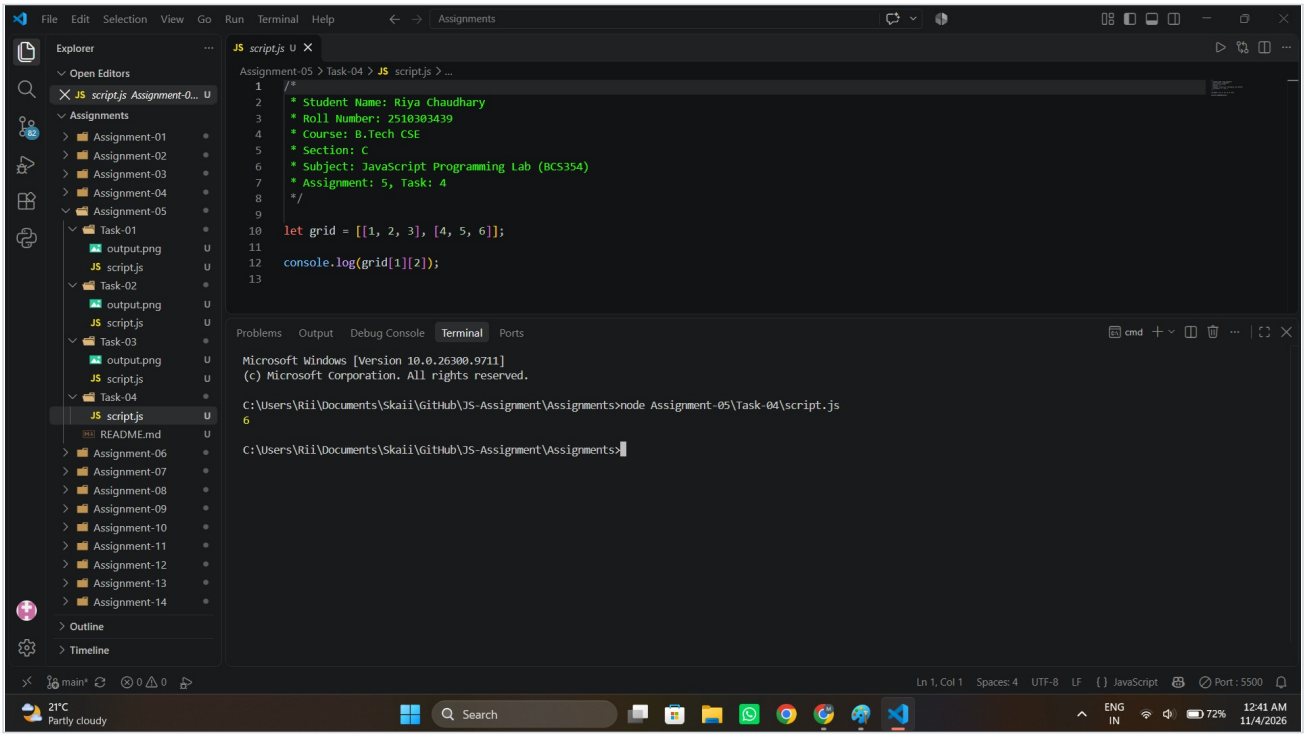
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 5, Task: 4
 */

let grid = [[1, 2, 3], [4, 5, 6]];

console.log(grid[1][2]);
```

Execution Output Screenshot:



Assignment 6: JSON Text

Assignment Aim / Objective: To convert an object to JSON text, convert JSON text back to an object, and catch invalid JSON.

Task 1: stringify

Problem Statement: Create an object with name Rohan and marks [70, 80]. Convert it with JSON.stringify. Print the text.

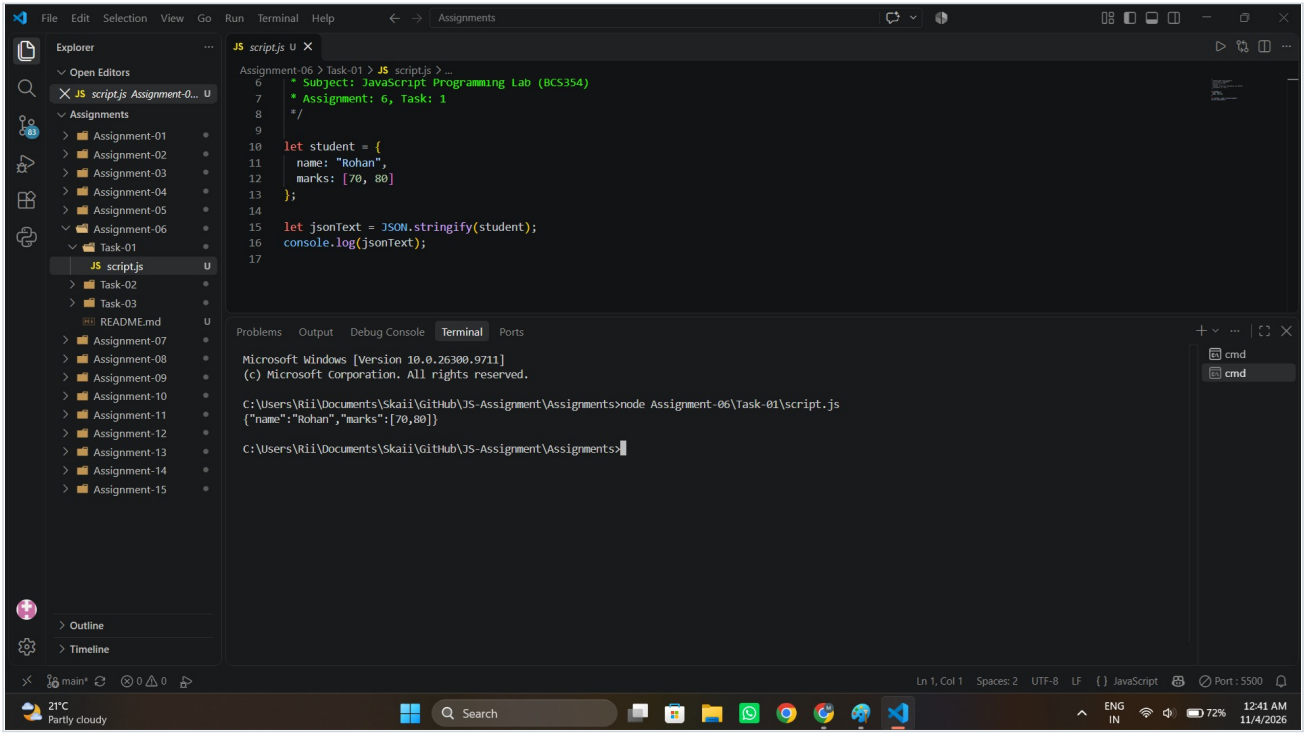
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 6, Task: 1
 */

let student = {
  name: "Rohan",
  marks: [70, 80]
};

let jsonText = JSON.stringify(student);
console.log(jsonText);
```

Execution Output Screenshot:



Task 2: parse

Problem Statement: Store the text {"city":"Pune","pin":411001}. Convert it with JSON.parse. Print the city.

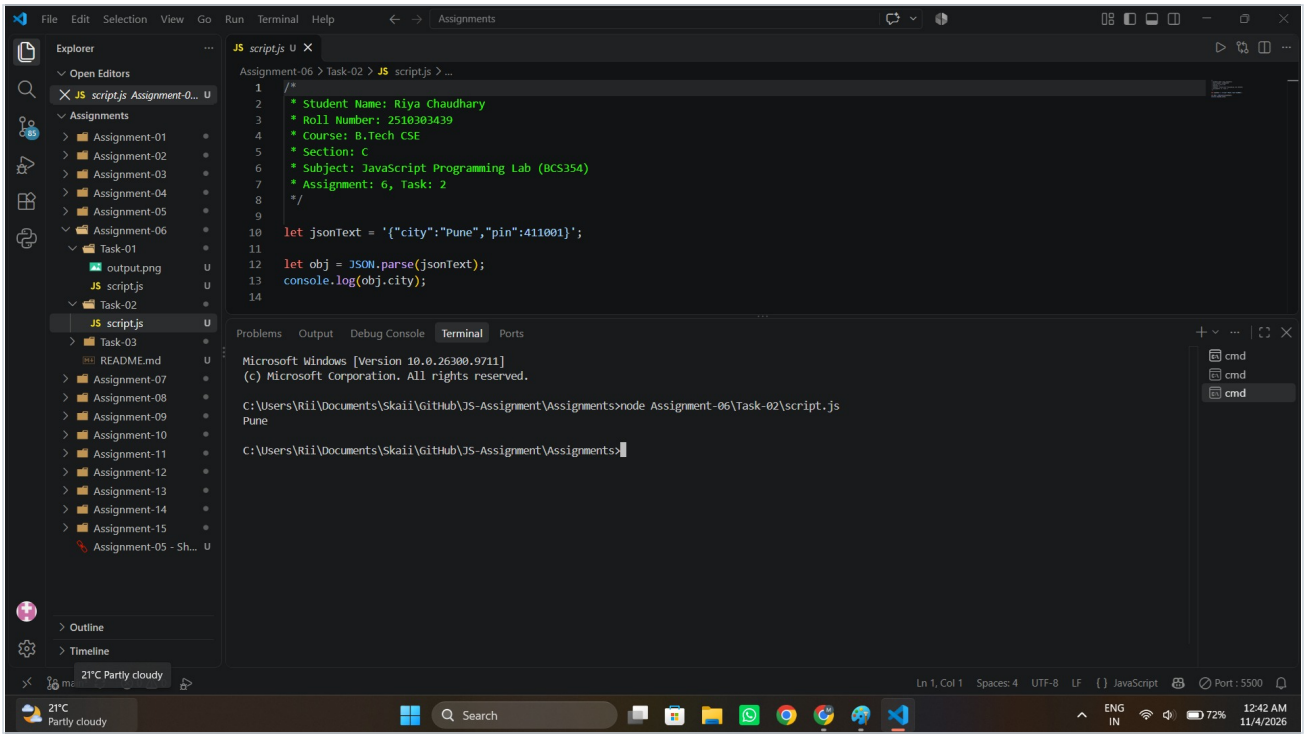
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 6, Task: 2
 */

let jsonText = '{"city":"Pune","pin":411001}';

let obj = JSON.parse(jsonText);
console.log(obj.city);
```

Execution Output Screenshot:



Task 3: Invalid JSON with try and catch

Problem Statement: Store the invalid text {name: Rohan}. Call JSON.parse inside a try block. Print Invalid JSON in the catch block.

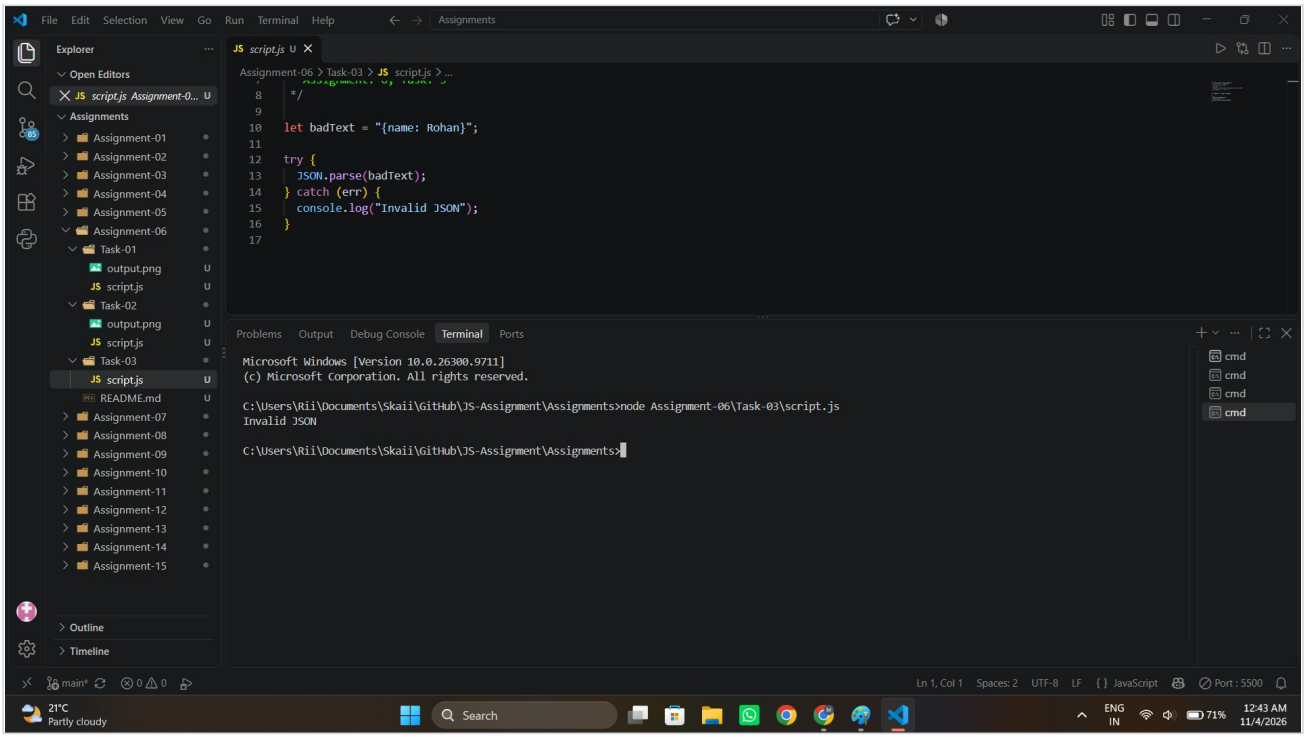
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 6, Task: 3
 */

let badText = "{name: Rohan}";

try {
  JSON.parse(badText);
} catch (err) {
  console.log("Invalid JSON");
}
```

Execution Output Screenshot:



Assignment 7: Constructor Functions and Classes

Assignment Aim / Objective: To create objects with a constructor function and with an ES6 class, and to add a method through the prototype.

Task 1: Constructor function

Problem Statement: Create function Student(name, marks) saving both on this. Create new Student("Asha", 88). Print the name and marks on separate lines.

Source Code (script.js):

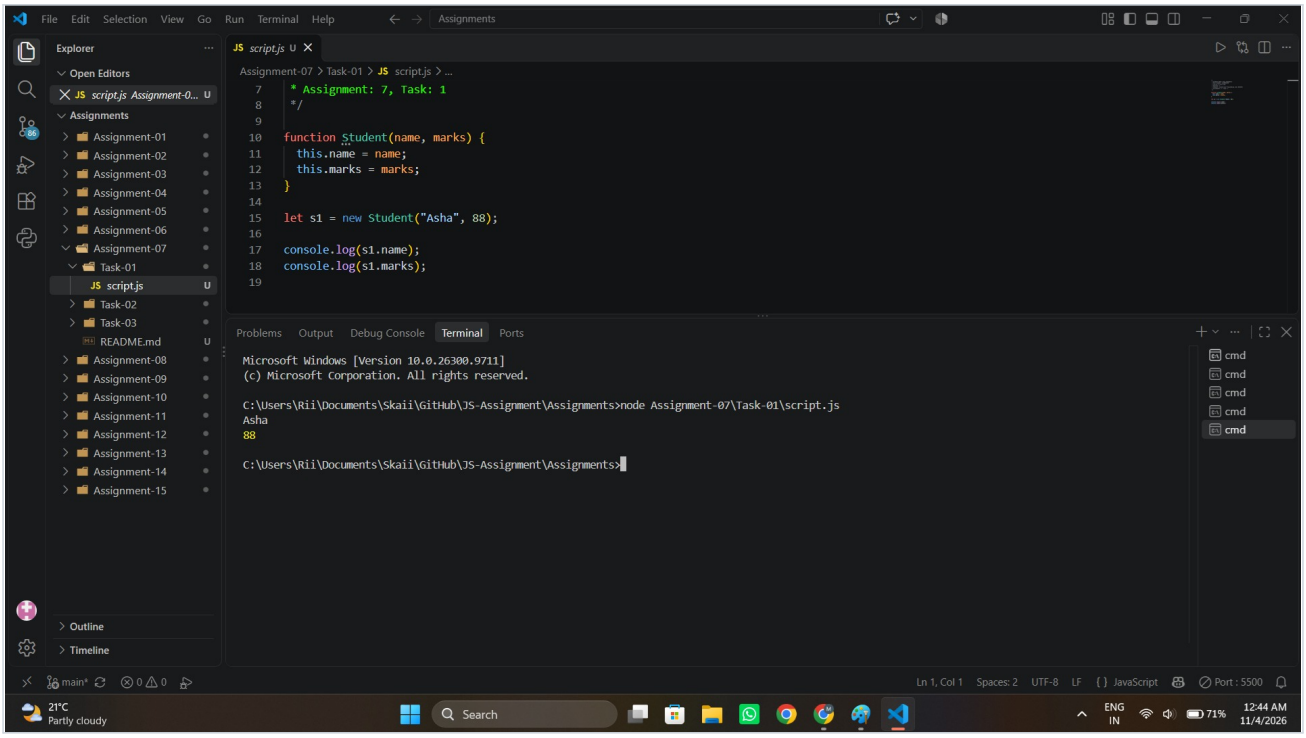
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 7, Task: 1
 */

function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}

let s1 = new Student("Asha", 88);

console.log(s1.name);
console.log(s1.marks);
```

Execution Output Screenshot:



Task 2: ES6 class

Problem Statement: Create class Book with a constructor(title, pages). Add method summary() returning title + " has " + pages + " pages". Create Book("Algorithms", 320) and print the summary.

Source Code (script.js):

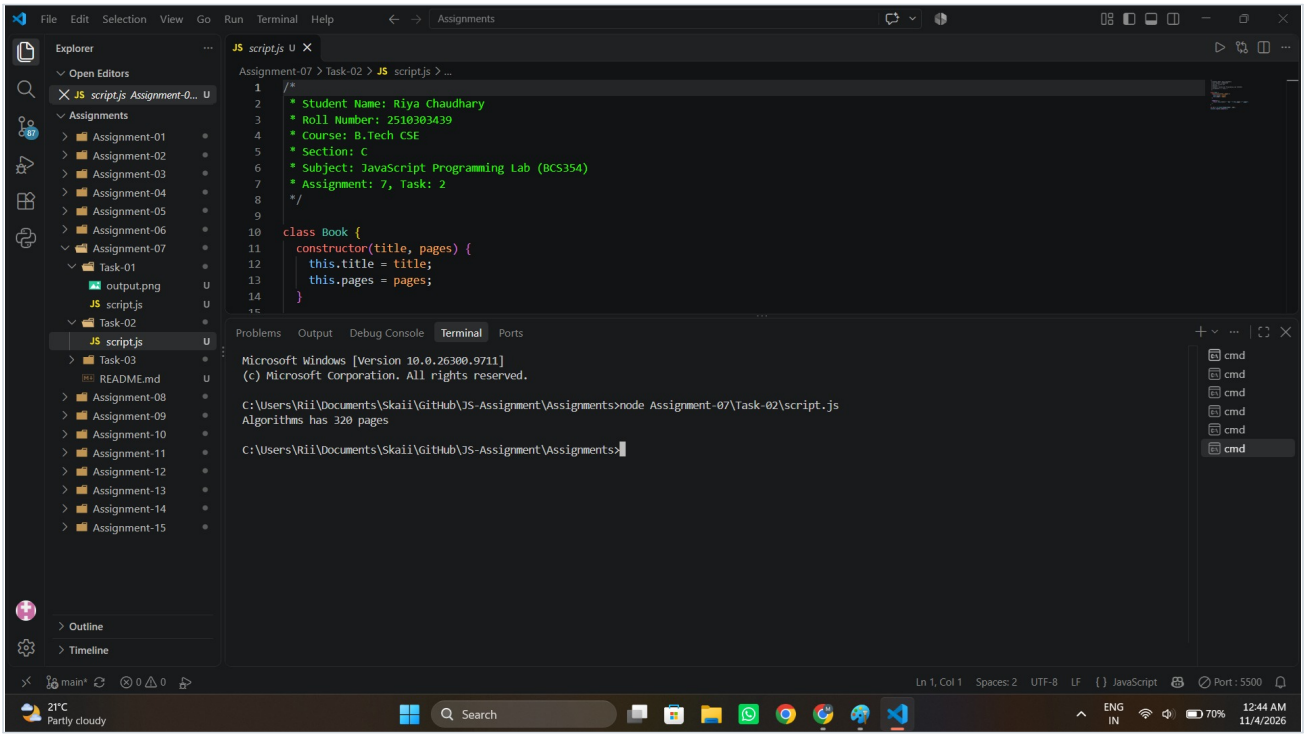
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 7, Task: 2
 */

class Book {
  constructor(title, pages) {
    this.title = title;
    this.pages = pages;
  }

  summary() {
    return this.title + " has " + this.pages + " pages";
  }
}

let b1 = new Book("Algorithms", 320);
console.log(b1.summary());
```

Execution Output Screenshot:



Task 3: Prototype method

Problem Statement: Create the constructor function Student(name, marks). Add result() to Student.prototype: Pass for marks >= 40, otherwise Fail. Create Asha with marks 88 and print the result.

Source Code (script.js):

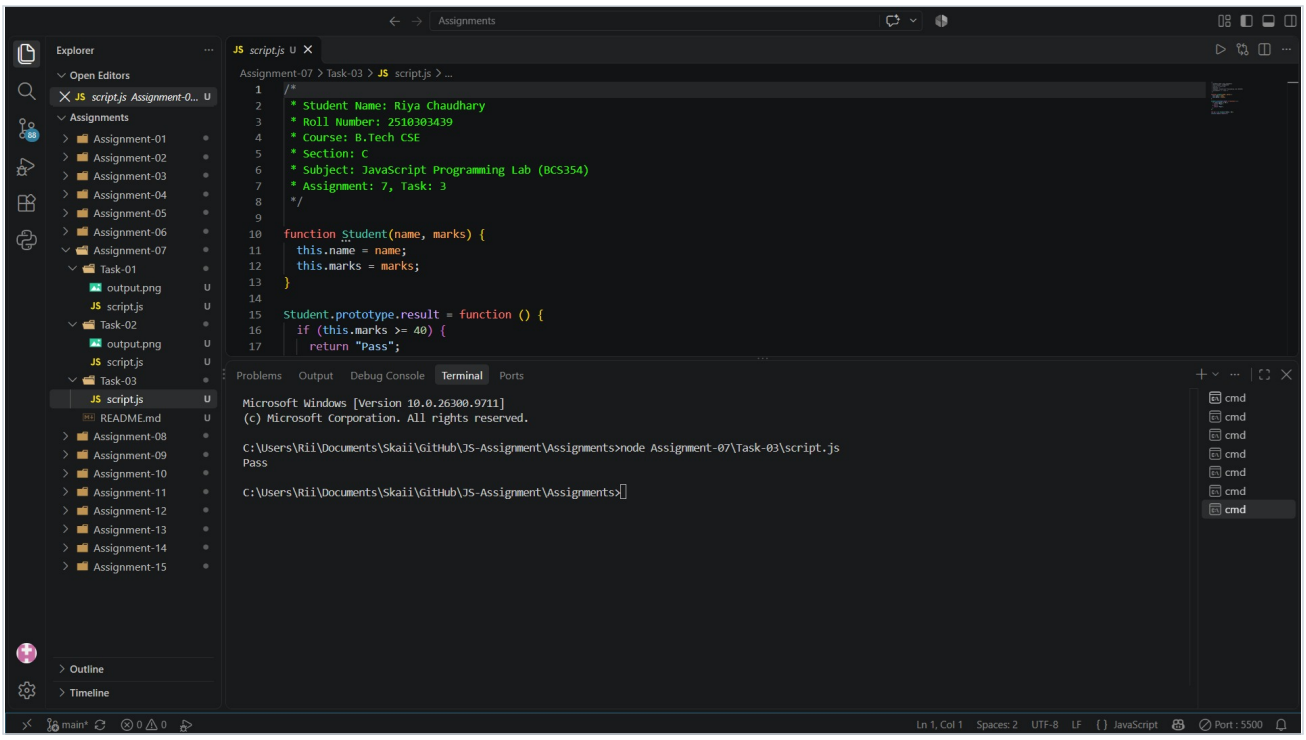
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 7, Task: 3
 */

function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}

Student.prototype.result = function () {
  if (this.marks >= 40) {
    return "Pass";
  } else {
    return "Fail";
  }
};

let s1 = new Student("Asha", 88);
console.log(s1.result());
```

Execution Output Screenshot:



Assignment 8: Inheritance, Methods and Composition

Assignment Aim / Objective: To use extends to make a child class, write a method that checks a value, and place one object inside another.

Task 1: Inheritance with extends

Problem Statement: Create class Person with greet() returning "Hi, I am " + name. Create class Teacher extends Person with a subject. Create Teacher Anita (Maths) and print her greeting.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 8, Task: 1
 */

class Person {
  constructor(name) {
    this.name = name;
  }

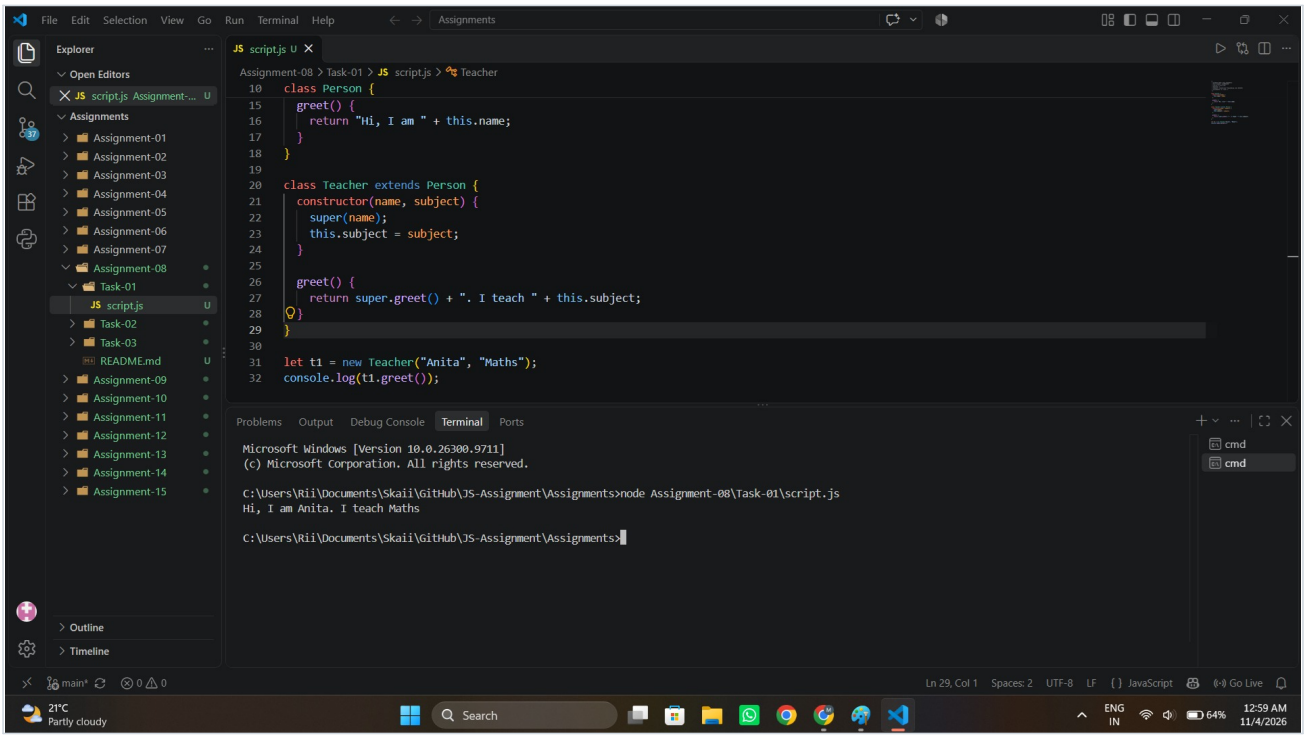
  greet() {
    return "Hi, I am " + this.name;
  }
}

class Teacher extends Person {
  constructor(name, subject) {
    super(name);
    this.subject = subject;
  }

  greet() {
    return super.greet() + ". I teach " + this.subject;
  }
}

let t1 = new Teacher("Anita", "Maths");
console.log(t1.greet());
```

Execution Output Screenshot:



Task 2: Locker with checkPin

Problem Statement: Create class Locker with the pin 1234. Add checkPin(number) returning true if number equals the pin, otherwise false. Print checkPin(1111) and checkPin(1234).

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 8, Task: 2
 */

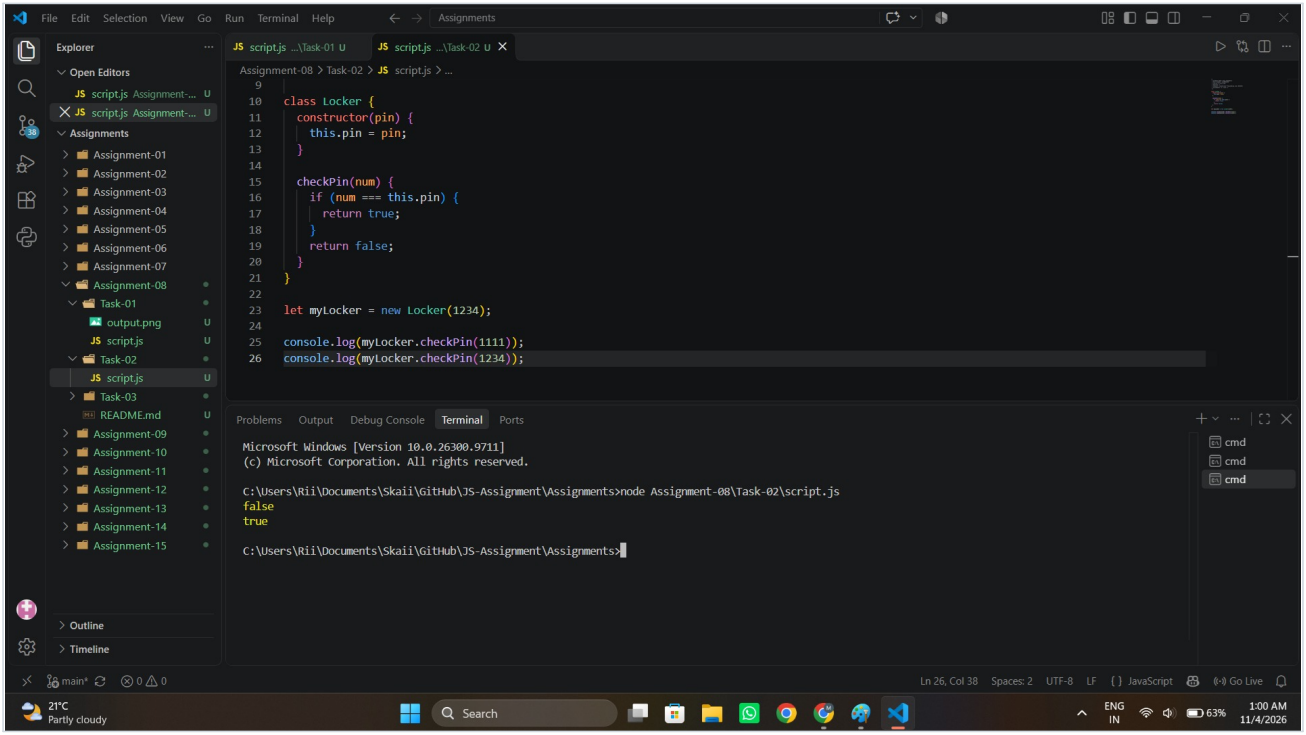
class Locker {
  constructor(pin) {
    this.pin = pin;
  }

  checkPin(num) {
    if (num === this.pin) {
      return true;
    }
    return false;
  }
}

let myLocker = new Locker(1234);

console.log(myLocker.checkPin(1111));
console.log(myLocker.checkPin(1234));
```

Execution Output Screenshot:



Task 3: Composition: Car has an Engine

Problem Statement: Create class Engine that stores a power. Create class Car that builds an Engine from its power. Add start() returning "Car starts with " + power + " HP". Create Car(150) and print start().

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 8, Task: 3
 */

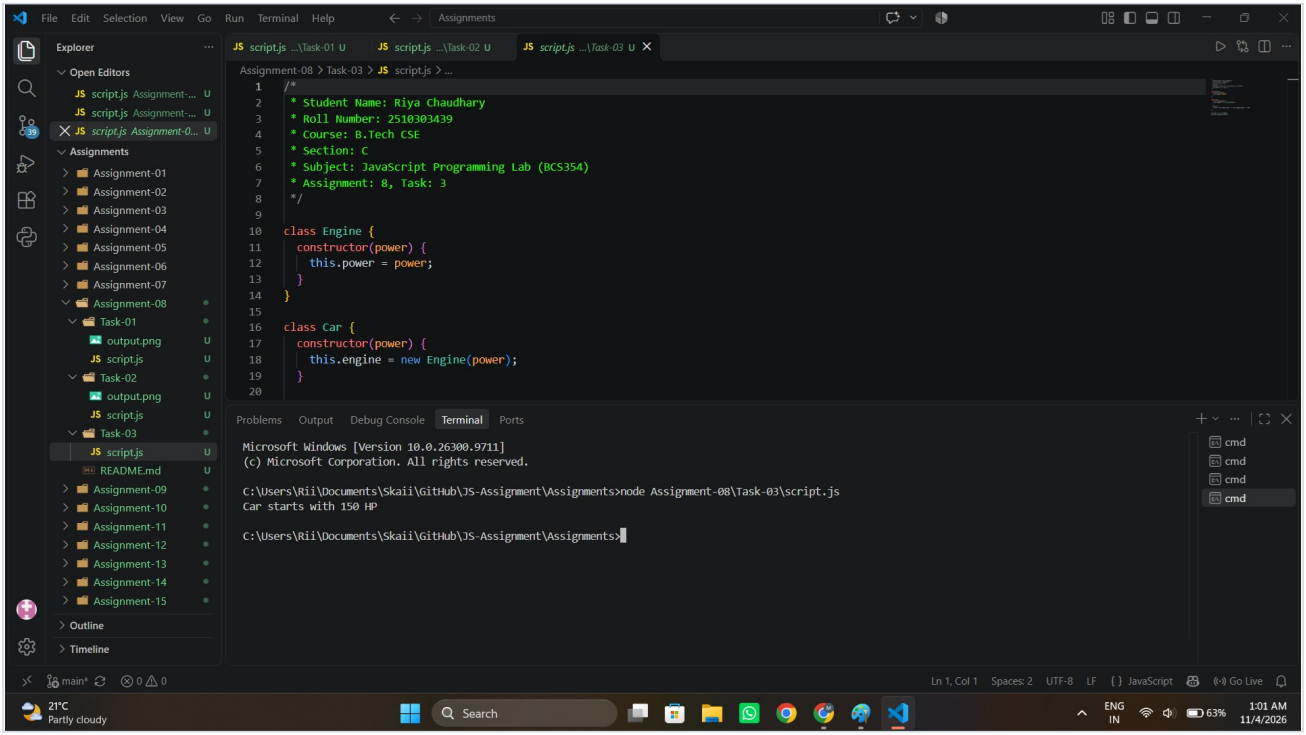
class Engine {
  constructor(power) {
    this.power = power;
  }
}

class Car {
  constructor(power) {
    this.engine = new Engine(power);
  }

  start() {
    return "Car starts with " + this.engine.power + " HP";
  }
}

let car1 = new Car(150);
console.log(car1.start());
```

Execution Output Screenshot:



Assignment 9: DOM, Validation, Storage and Quiz

Assignment Aim / Objective: To change a page with a click, check form values in code, save text in local storage, and build a small quiz.

Task 1: Change heading on click

Problem Statement: Create a heading Welcome and a button Change text. On click, change the heading text to Button clicked.

Source Code (index.html):

```
<!--
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 1
-->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Change Text</title>
</head>
<body>
<h1 id="heading">Welcome</h1>
<button id="changeBtn">Change text</button>
<script src="script.js"></script>
</body>
</html>
```

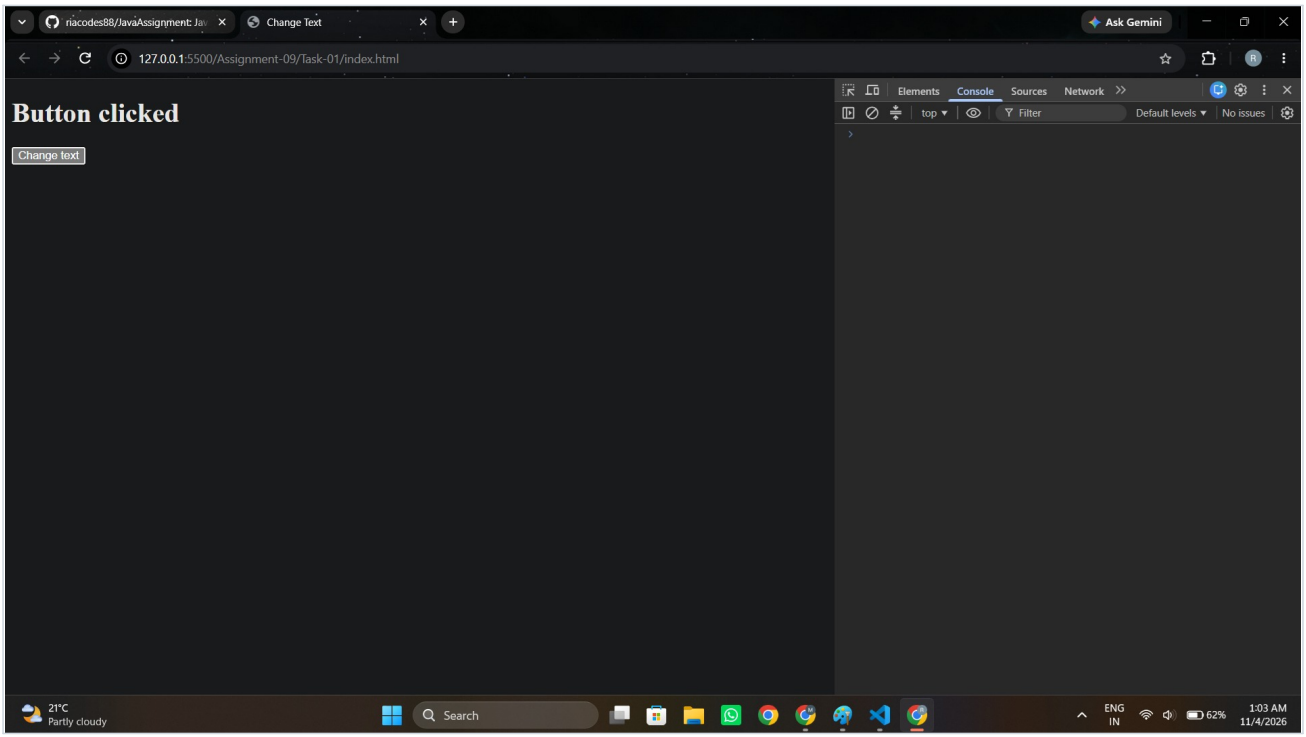
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 1
 */

let heading = document.getElementById("heading");
let btn = document.getElementById("changeBtn");

btn.addEventListener("click", function () {
  heading.textContent = "Button clicked";
});
```

Execution Output Screenshot:



Task 2: validateForm function

Problem Statement: Create validateForm(name) that makes an empty list errors. If name is empty, add "Name is required" to the list. Return the list. Print validateForm("Asha") and validateForm("").

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 2
 */

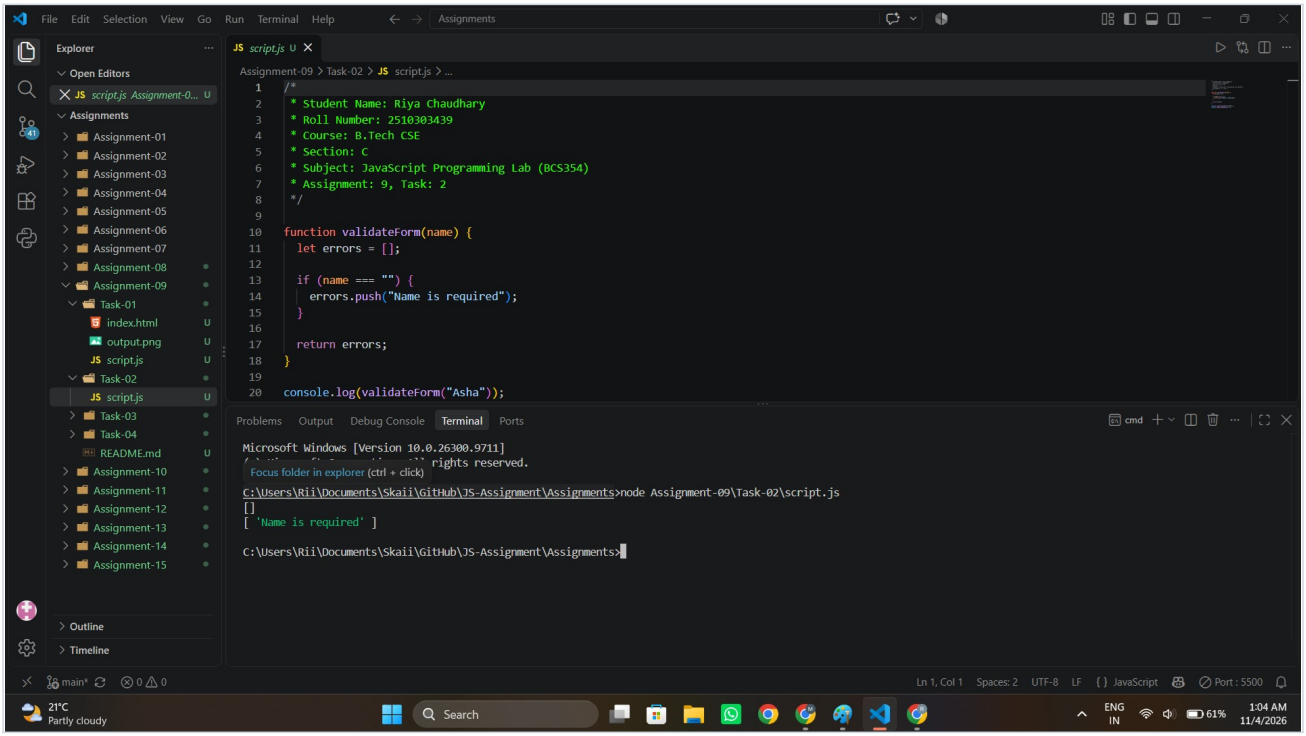
function validateForm(name) {
  let errors = [];

  if (name === "") {
    errors.push("Name is required");
  }

  return errors;
}

console.log(validateForm("Asha"));
console.log(validateForm(""));
```

Execution Output Screenshot:



Task 3: Save a note in localStorage

Problem Statement: Create a text box and a Save button. On Save, store the text in localStorage with the key note. When the page opens, read the key note and show it in the box.

Source Code (index.html):

```
<!--
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 3
-->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Local Storage Note</title>
</head>
<body>
<input type="text" id="noteBox">
<button id="saveBtn">Save</button>
<script src="script.js"></script>
</body>
</html>
```

Source Code (script.js):

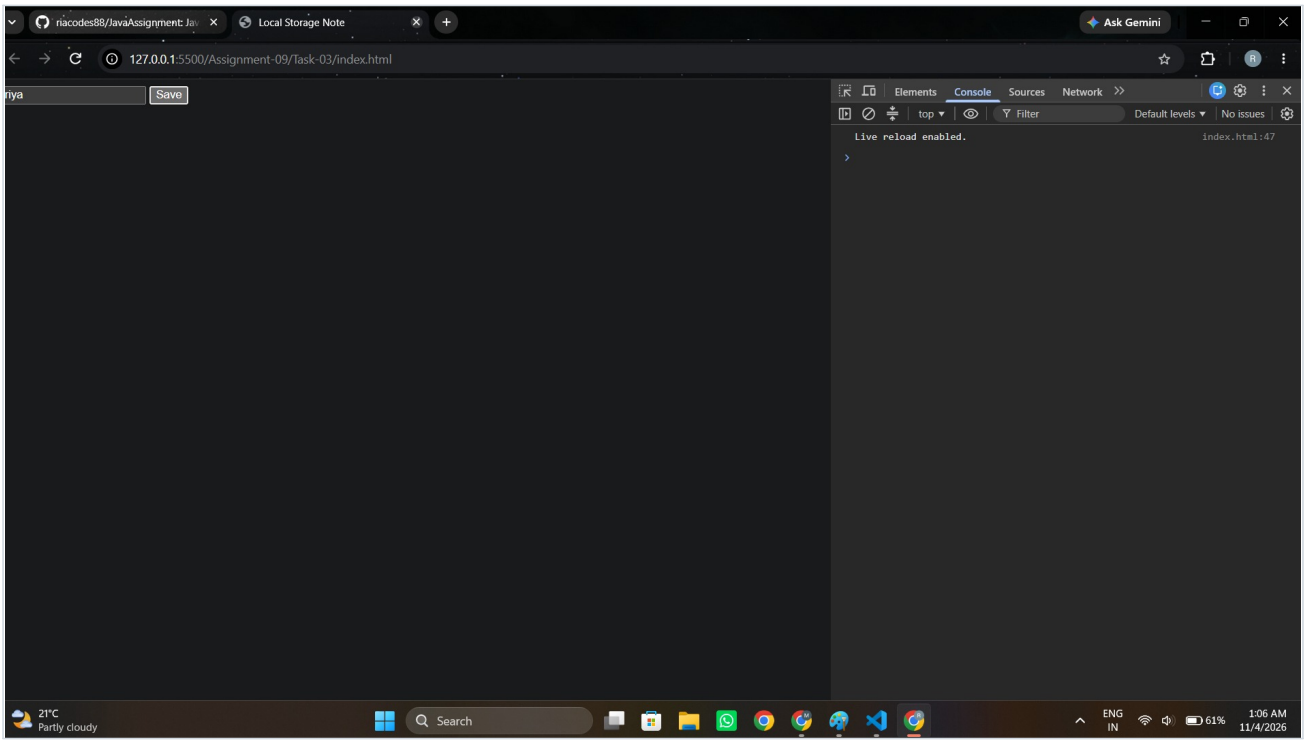
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 3
 */

let noteBox = document.getElementById("noteBox");
let saveBtn = document.getElementById("saveBtn");

let savedNote = localStorage.getItem("note");
if (savedNote) {
  noteBox.value = savedNote;
}

saveBtn.addEventListener("click", function () {
  localStorage.setItem("note", noteBox.value);
});
```

Execution Output Screenshot:



Task 4: Quiz

Problem Statement: Show the question What is 2 + 3? with radio buttons 4, 5 and 6. On Check answer, find the selected radio button. Show Correct if the answer is 5, otherwise Try again.

Source Code (index.html):

```
<!--
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 4
-->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Quiz</title>
</head>
<body>
<p>What is 2 + 3?</p>
<label><input type="radio" name="answer" value="4"> 4</label>
<label><input type="radio" name="answer" value="5"> 5</label>
<label><input type="radio" name="answer" value="6"> 6</label>
<br><br>
<button id="checkBtn">Check answer</button>
<p id="result"></p>
<script src="script.js"></script>
</body>
</html>
```

Source Code (script.js):

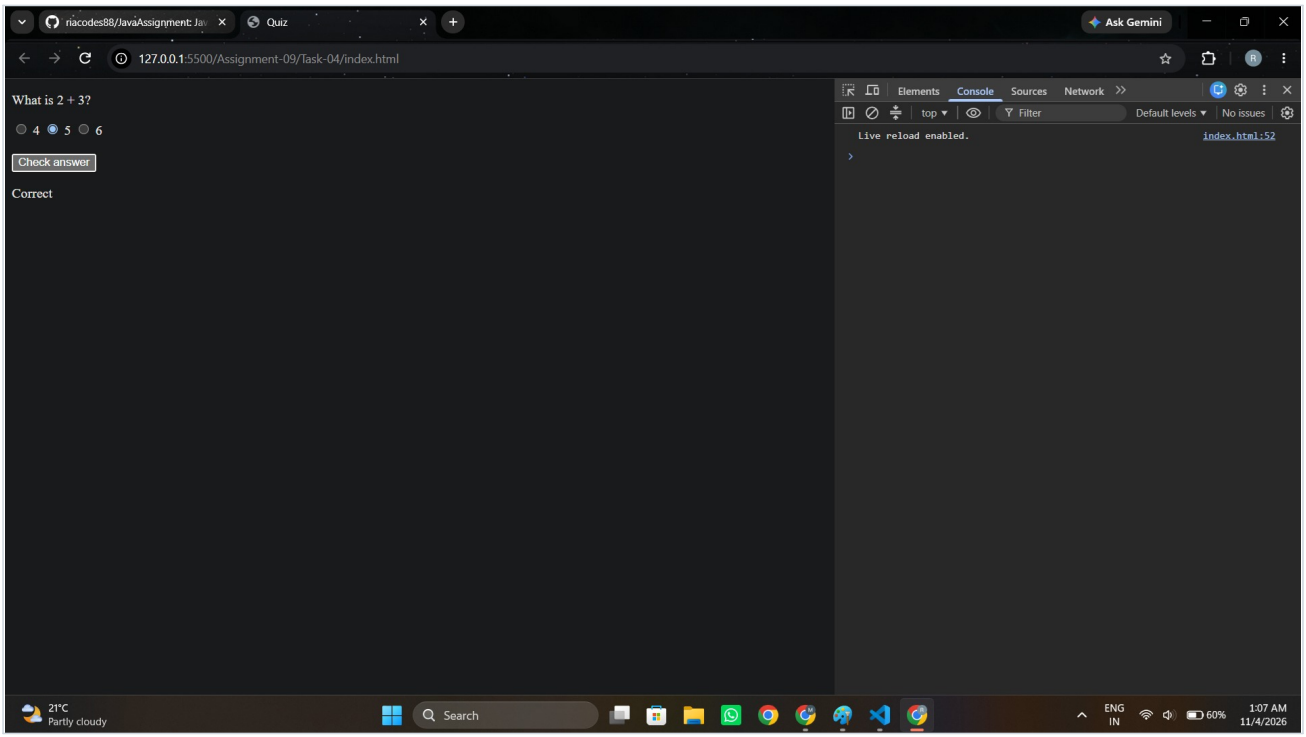
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 9, Task: 4
 */

let checkBtn = document.getElementById("checkBtn");
let result = document.getElementById("result");

checkBtn.addEventListener("click", function () {
  let chosen = document.querySelector('input[name="answer"]:checked');

  if (chosen && chosen.value === "5") {
    result.textContent = "Correct";
  } else {
    result.textContent = "Try again";
  }
});
```

Execution Output Screenshot:



Assignment 10: Objects and Destructuring

Assignment Aim / Objective: To create an object, read and change its properties, and take values out of an array and an object with destructuring.

Task 1: Object properties

Problem Statement: Create the object student with name, roll and marks. Print the name. Change marks to 81 and print marks.

Source Code (script.js):

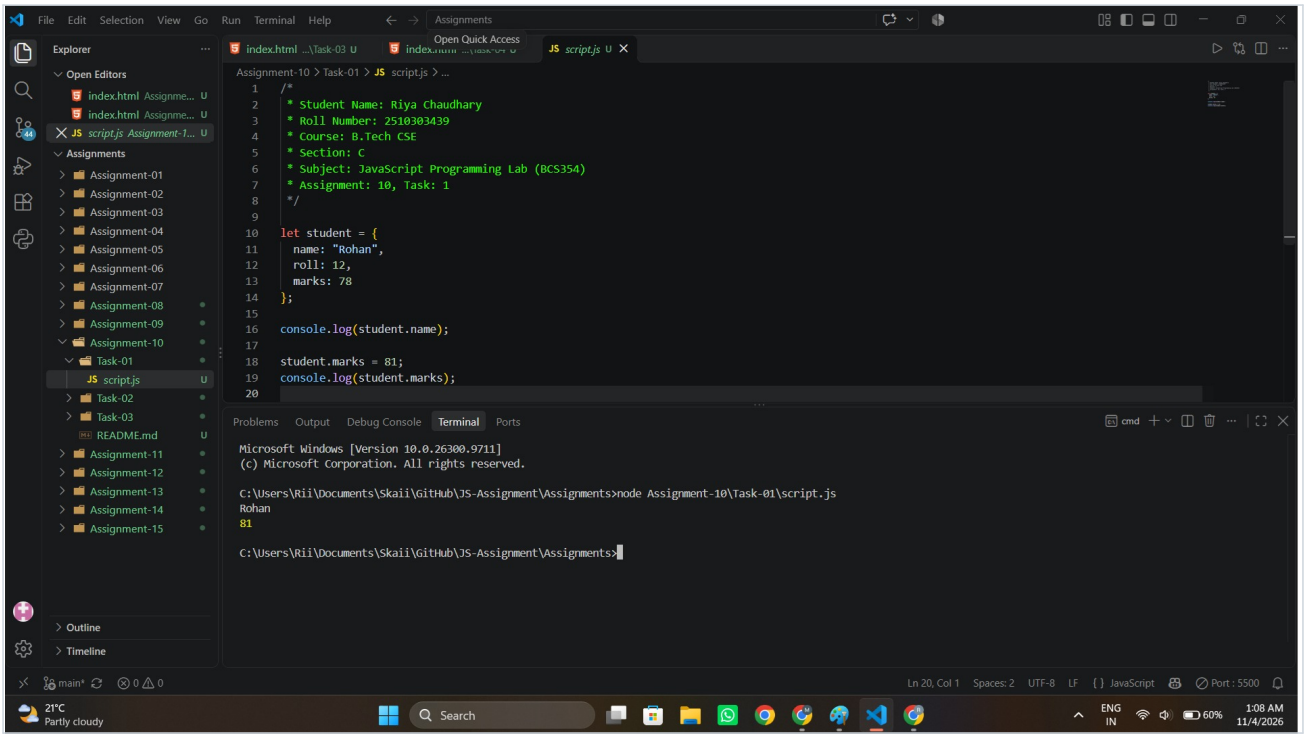
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 10, Task: 1
 */

let student = {
  name: "Rohan",
  roll: 12,
  marks: 78
};

console.log(student.name);

student.marks = 81;
console.log(student.marks);
```

Execution Output Screenshot:



Task 2: Array destructuring

Problem Statement: Store Maths, Science and English in an array. Use const [first, second] = subjects. Print first and second on separate lines.

Source Code (script.js):

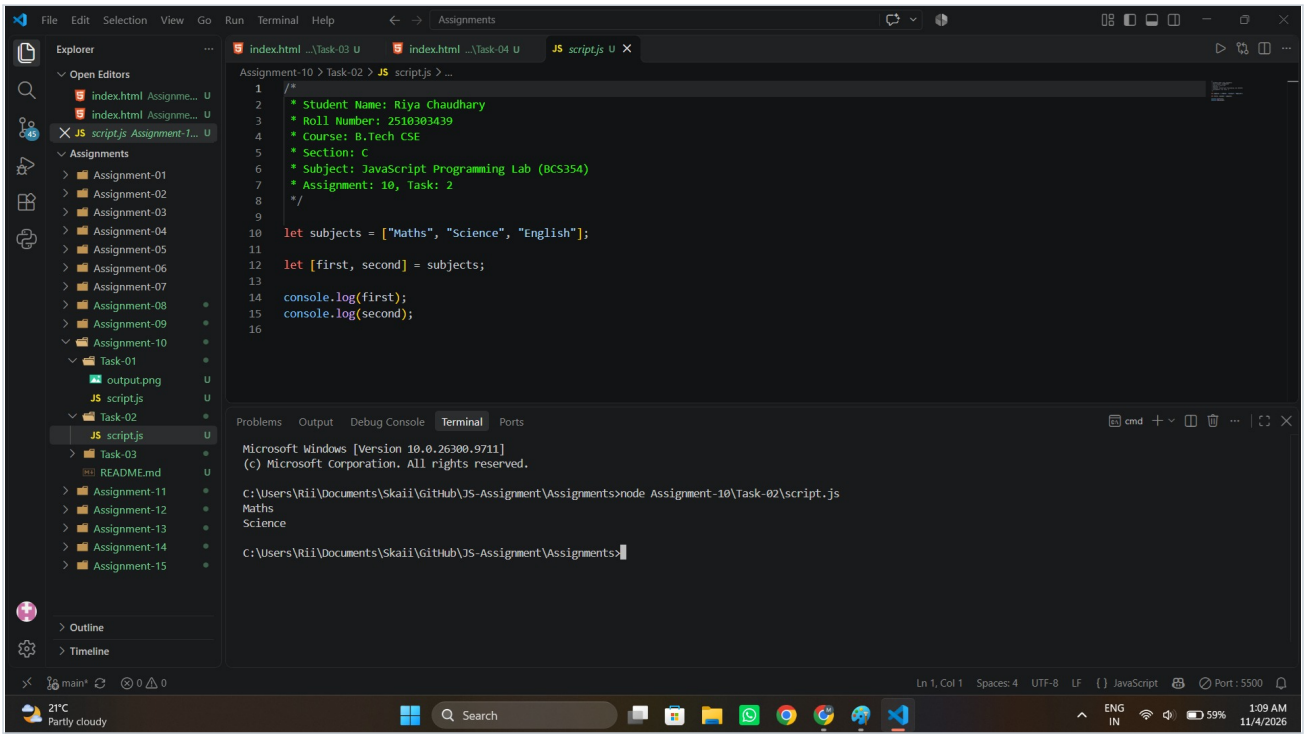
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 10, Task: 2
 */

let subjects = ["Maths", "Science", "English"];

let [first, second] = subjects;

console.log(first);
console.log(second);
```

Execution Output Screenshot:



Task 3: Object destructuring

Problem Statement: Create student with name Rohan, roll 12 and marks 81. Use const { name, roll } = student. Print name and roll on separate lines.

Source Code (script.js):

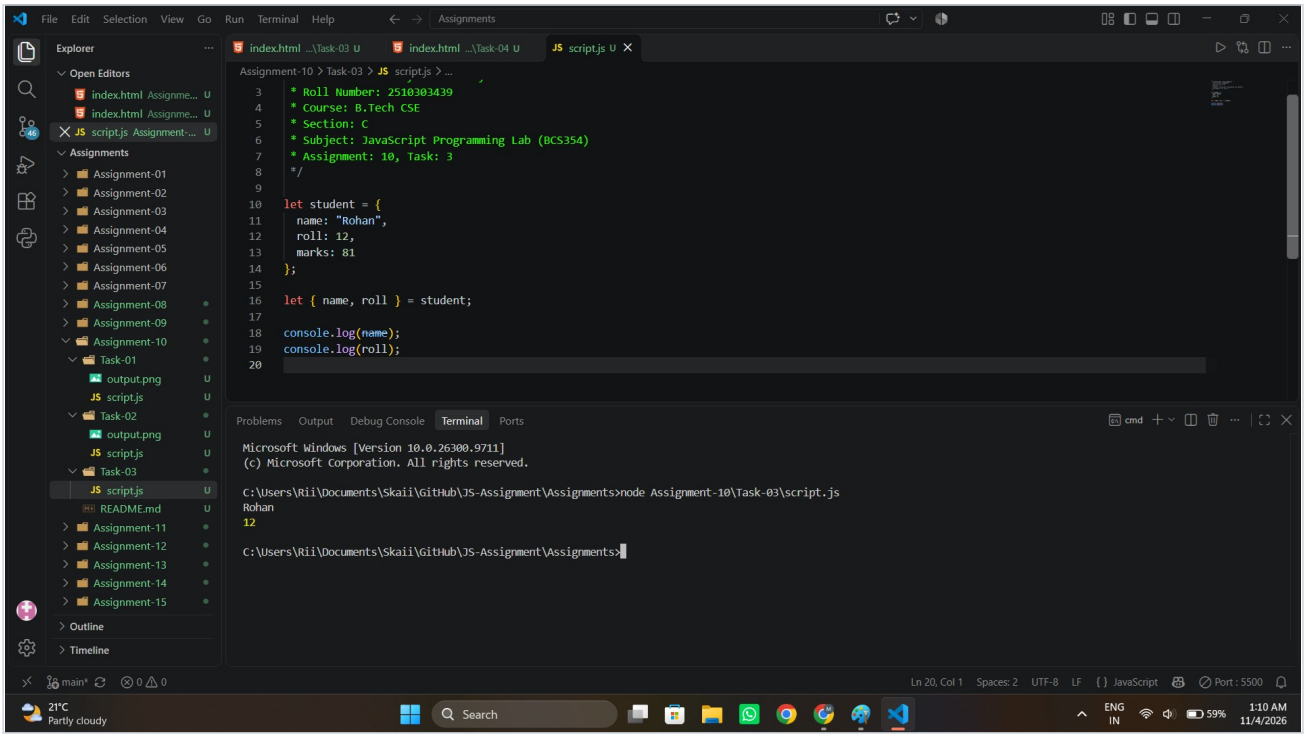
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 10, Task: 3
 */

let student = {
  name: "Rohan",
  roll: 12,
  marks: 81
};

let { name, roll } = student;

console.log(name);
console.log(roll);
```

Execution Output Screenshot:



Assignment 11: Scope and Closures

Assignment Aim / Objective: To explain block scope and global scope, and to use a function that returns a function that keeps a value between calls.

Task 1: Block scope

Problem Statement: Declare let level = "outer". Inside a block declare let level = "inner" and print it. After the block print level again.

Source Code (script.js):

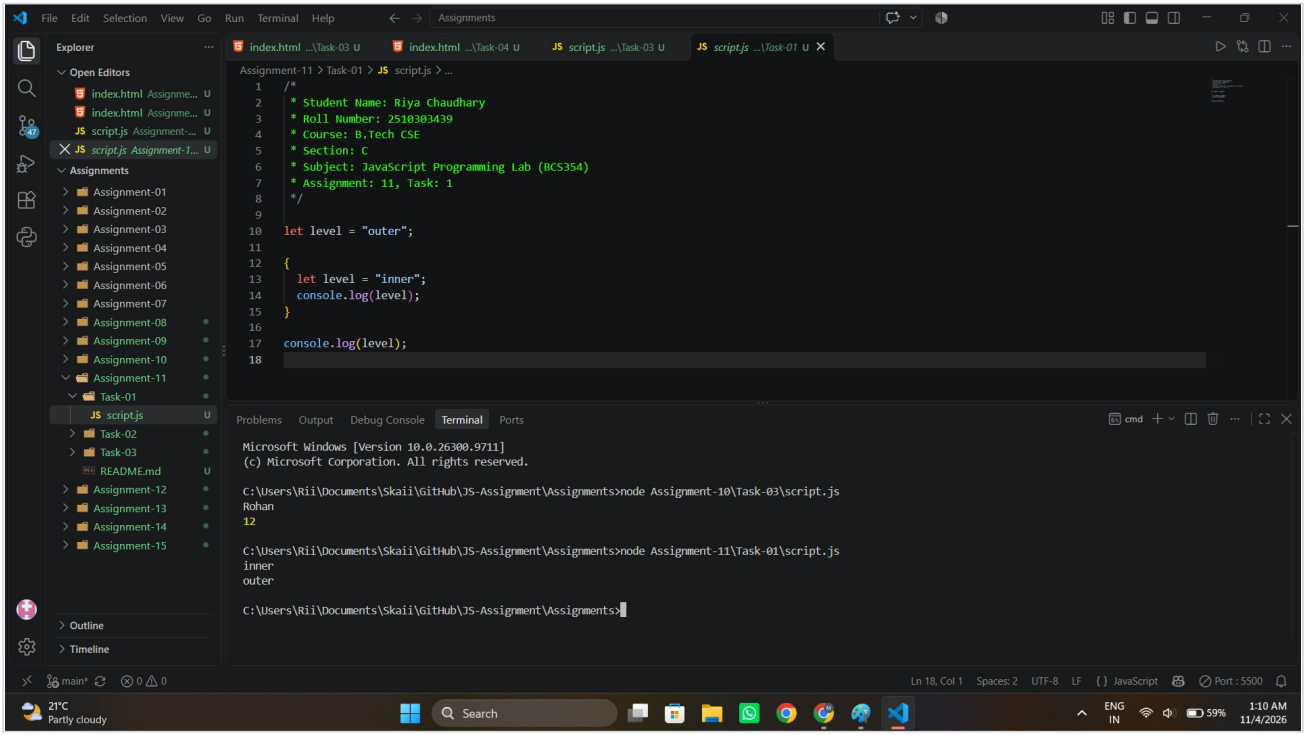
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 11, Task: 1
 */

let level = "outer";

{
  let level = "inner";
  console.log(level);
}

console.log(level);
```

Execution Output Screenshot:



Task 2: Closure counter

Problem Statement: Create makeCounter() that sets count = 0. Return a function that adds 1 to count and returns it. Call the returned function twice and print each result.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 11, Task: 2
 */

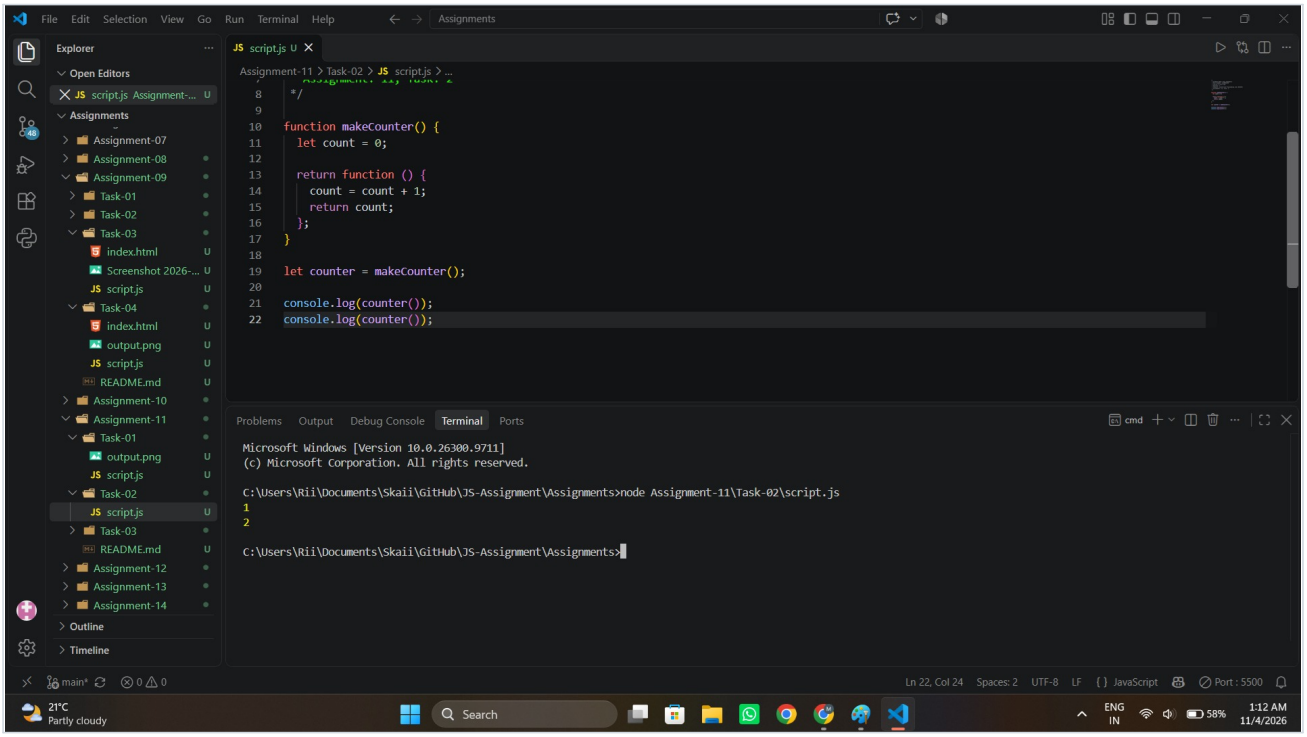
function makeCounter() {
  let count = 0;

  return function () {
    count = count + 1;
    return count;
  };
}

let counter = makeCounter();

console.log(counter());
console.log(counter());
```

Execution Output Screenshot:



Task 3: Global scope

Problem Statement: Declare const school = "Sample College" outside any function. Create showSchool() that prints "Welcome to " + school. Call showSchool().

Source Code (script.js):

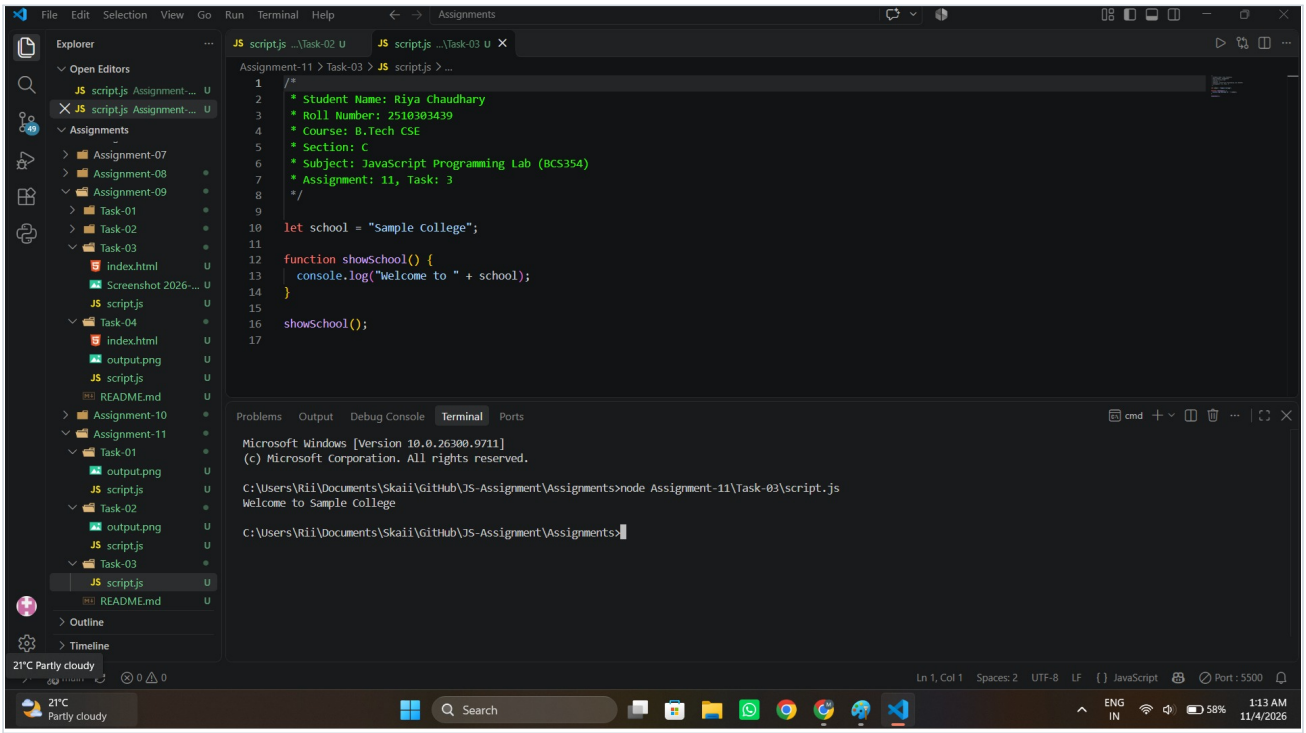
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 11, Task: 3
 */

let school = "Sample College";

function showSchool() {
  console.log("Welcome to " + school);
}

showSchool();
```

Execution Output Screenshot:



Assignment 12: Callbacks and Higher-Order Functions

Assignment Aim / Objective: To pass a function as an argument to another function and call it.

Task 1: applyTwice

Problem Statement: Create applyTwice(fn, value) returning fn(fn(value)). Create addTen(n) returning n + 10. Print applyTwice(addTen, 5).

Source Code (script.js):

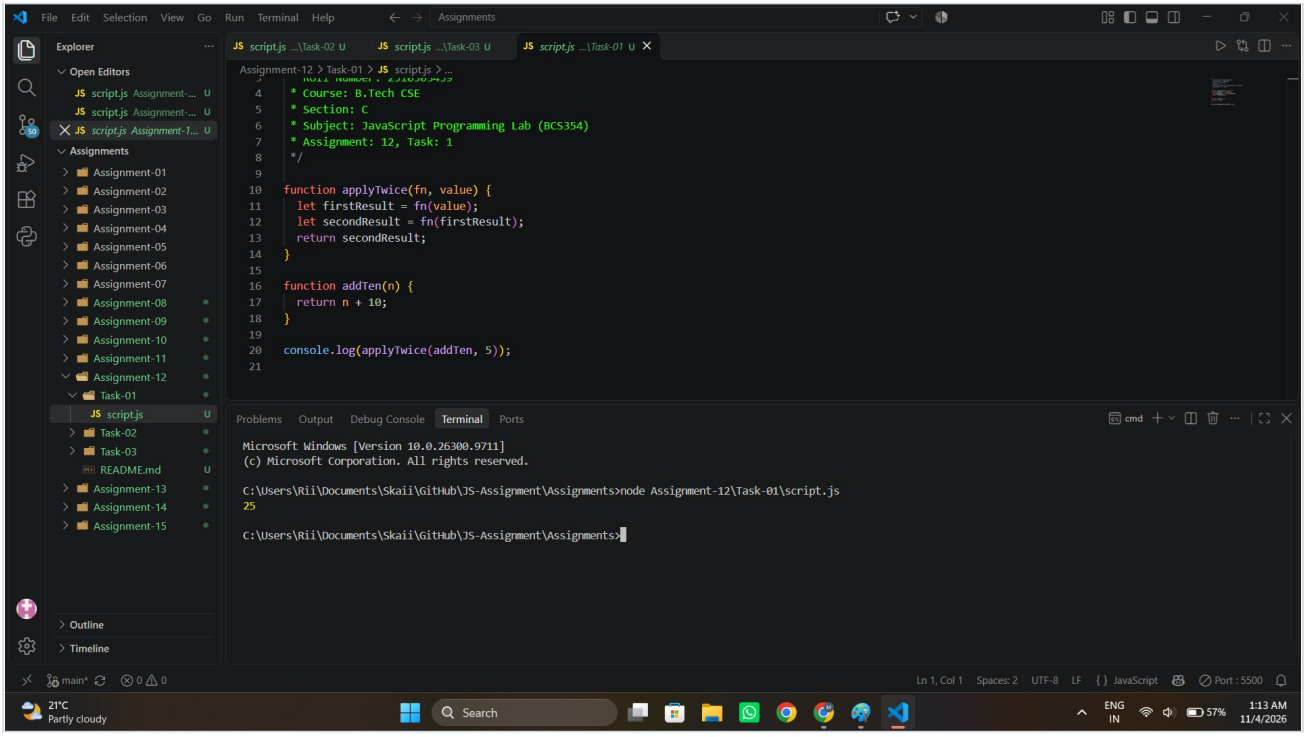
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 12, Task: 1
 */

function applyTwice(fn, value) {
  let firstResult = fn(value);
  let secondResult = fn(firstResult);
  return secondResult;
}

function addTen(n) {
  return n + 10;
}

console.log(applyTwice(addTen, 5));
```

Execution Output Screenshot:



Task 2: formatMarks with a callback

Problem Statement: Create formatMarks(marks, callback) returning callback(marks). Call it with 76 and an arrow function returning "Marks: " + marks + "/100". Print the result.

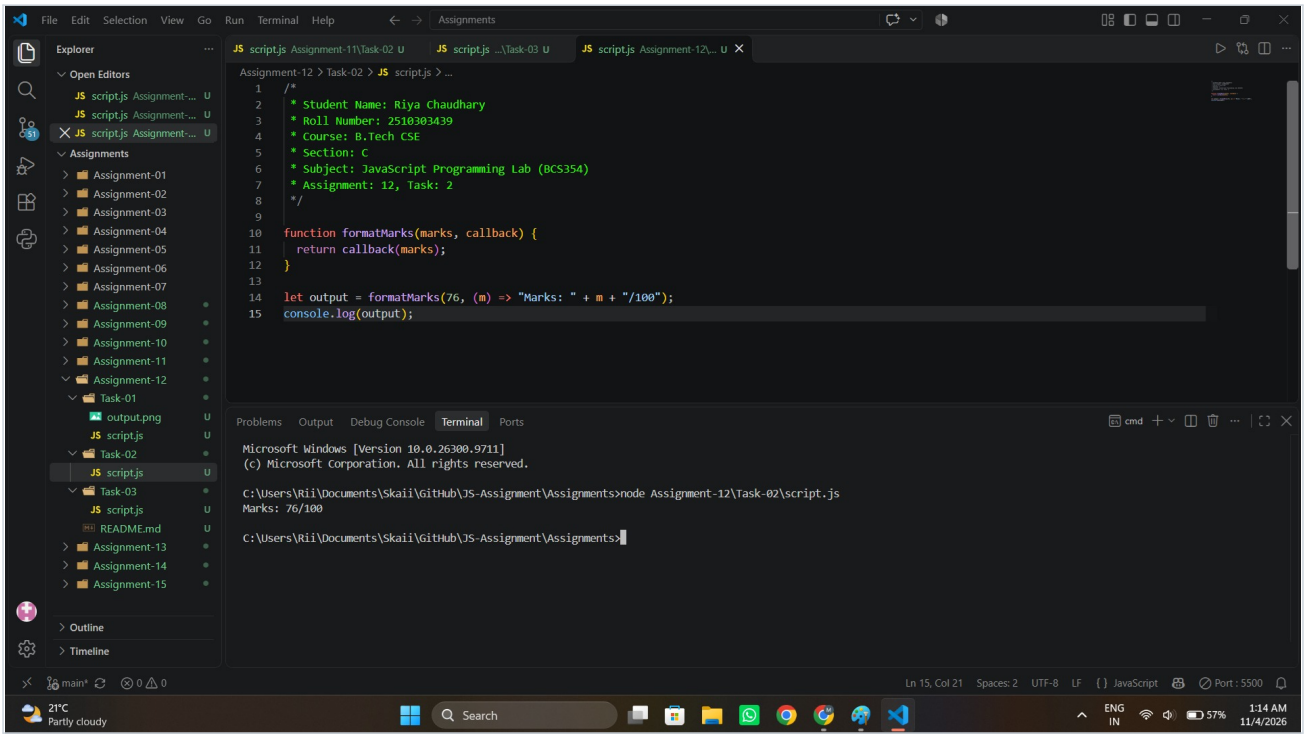
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 12, Task: 2
 */

function formatMarks(marks, callback) {
  return callback(marks);
}

let output = formatMarks(76, (m) => "Marks: " + m + "/100");
console.log(output);
```

Execution Output Screenshot:



Task 3: repeatAction

Problem Statement: Create repeatAction(times, action) that calls action(round) for round 1 to times. Call it with 3 and an arrow function printing "Round " + round.

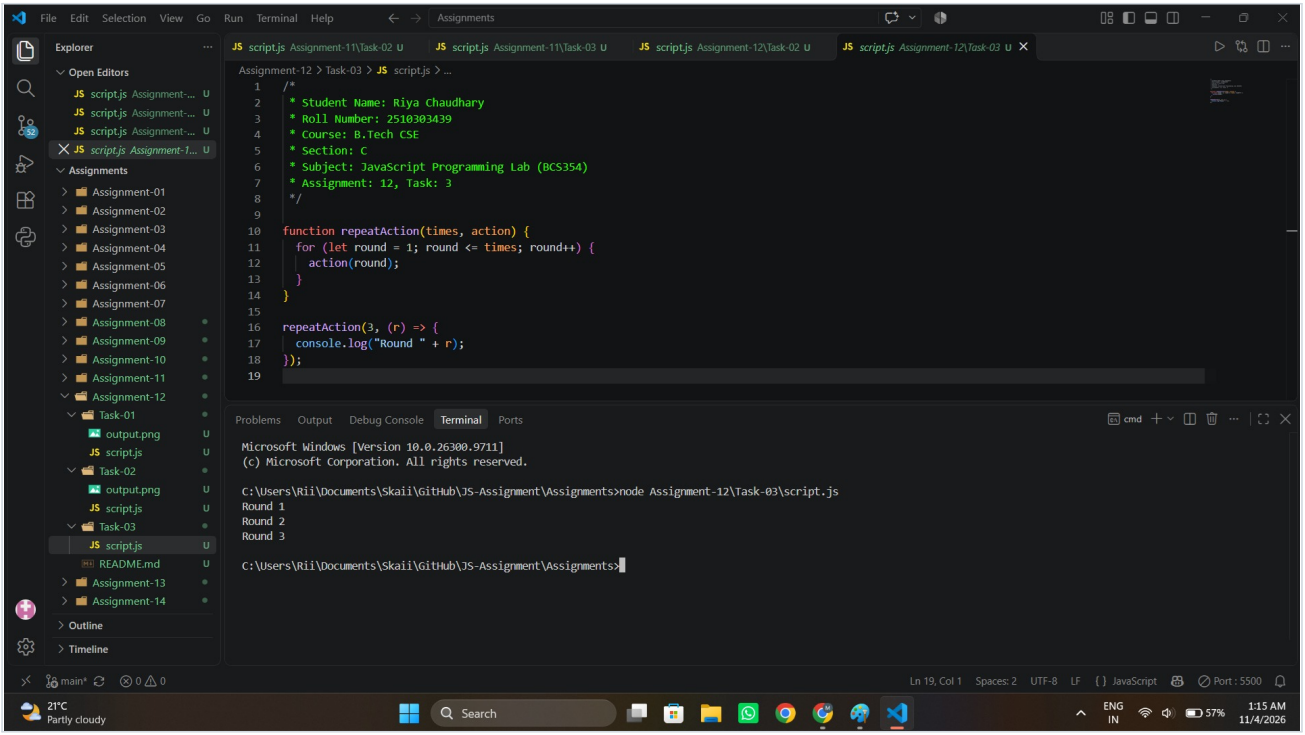
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 12, Task: 3
 */

function repeatAction(times, action) {
  for (let round = 1; round <= times; round++) {
    action(round);
  }
}

repeatAction(3, (r) => {
  console.log("Round " + r);
});
```

Execution Output Screenshot:



Assignment 13: Reduce, Some, Every and Map

Assignment Aim / Objective: To use reduce, some, every and map on arrays, and to keep the original array unchanged.

Task 1: reduce

Problem Statement: Store 120, 45 and 300 in an array. Use reduce starting from 0 to add the prices. Print the total.

Source Code (script.js):

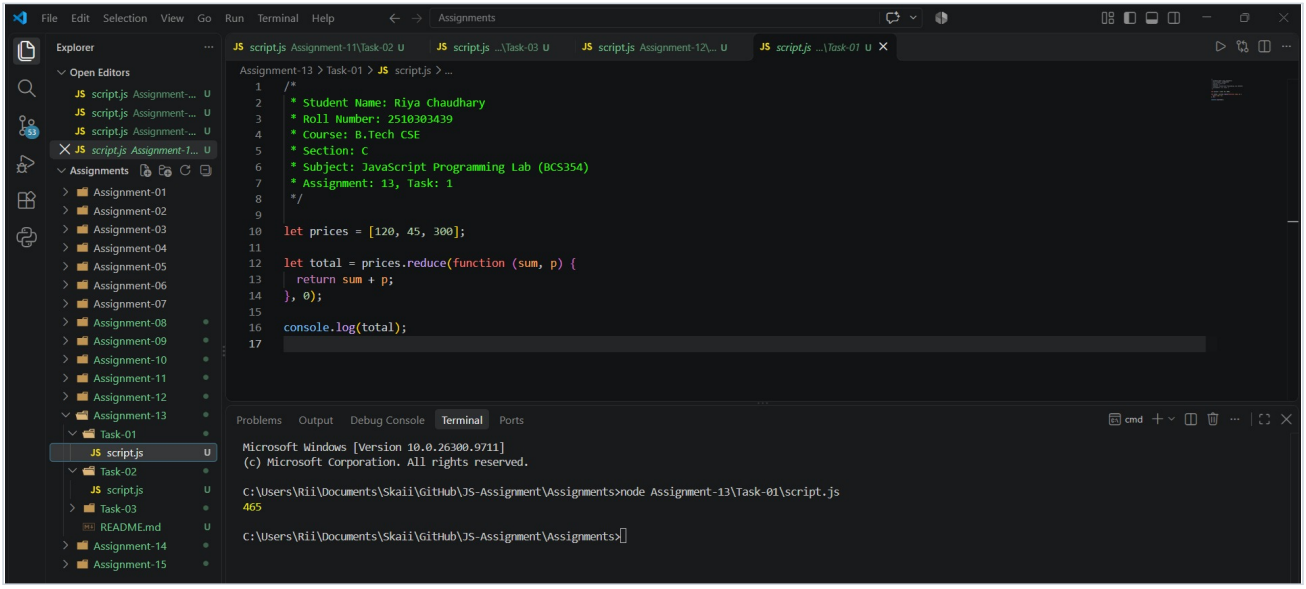
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 13, Task: 1
 */

let prices = [120, 45, 300];

let total = prices.reduce(function (sum, p) {
  return sum + p;
}, 0);

console.log(total);
```

Execution Output Screenshot:



Task 2: some and every

Problem Statement: Store 72, 85, 30 and 90 in an array. Use some to check whether any mark is below 40 and print the result. Use every to check whether all marks are 35 or more and print the result.

Source Code (script.js):

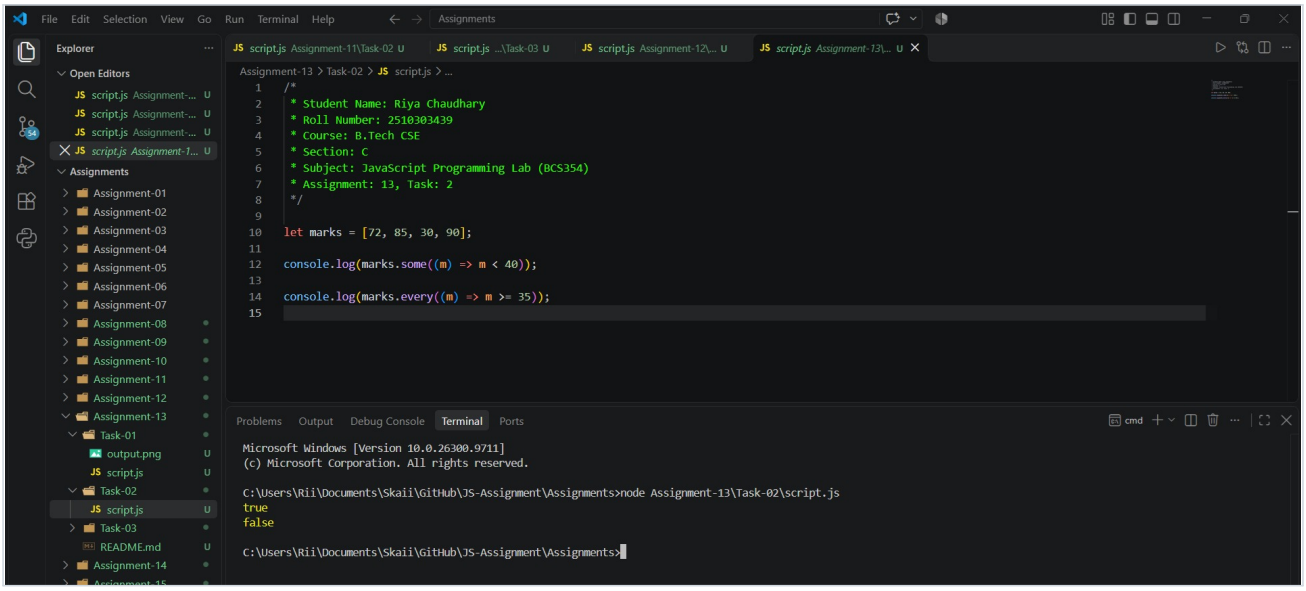
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 13, Task: 2
 */

let marks = [72, 85, 30, 90];

console.log(marks.some((m) => m < 40));

console.log(marks.every((m) => m >= 35));
```

Execution Output Screenshot:



Task 3: map

Problem Statement: Store 72, 85, 64 and 90 in an array. Use map to make a new array with every mark 5 more. Print the new array and then the original array.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 13, Task: 3
 */

let marks = [72, 85, 64, 90];

let newMarks = marks.map((m) => m + 5);

console.log(newMarks);
console.log(marks);
```

Execution Output Screenshot:

SCREENSHOT MISSING IN THE PROVIDED ZIP

Assignment-13 / Task-03 has no screenshot of its own (its *output.png* slot holds the Assignment 14 Task 1 screenshot). Run **script.js** and paste your terminal screenshot here before submitting.

Actual program output (verified by running *node script.js*):

```
[ 77, 90, 69, 95 ]
[ 72, 85, 64, 90 ]
```

Assignment 14: Timers and Intervals

Assignment Aim / Objective: To run code after a delay with setTimeout, cancel a timer, and repeat code with setInterval.

Task 1: setTimeout order

Problem Statement: Print Start. Schedule "Timer done" with setTimeout and delay 0. Print End. Note the order of the three lines.

Source Code (script.js):

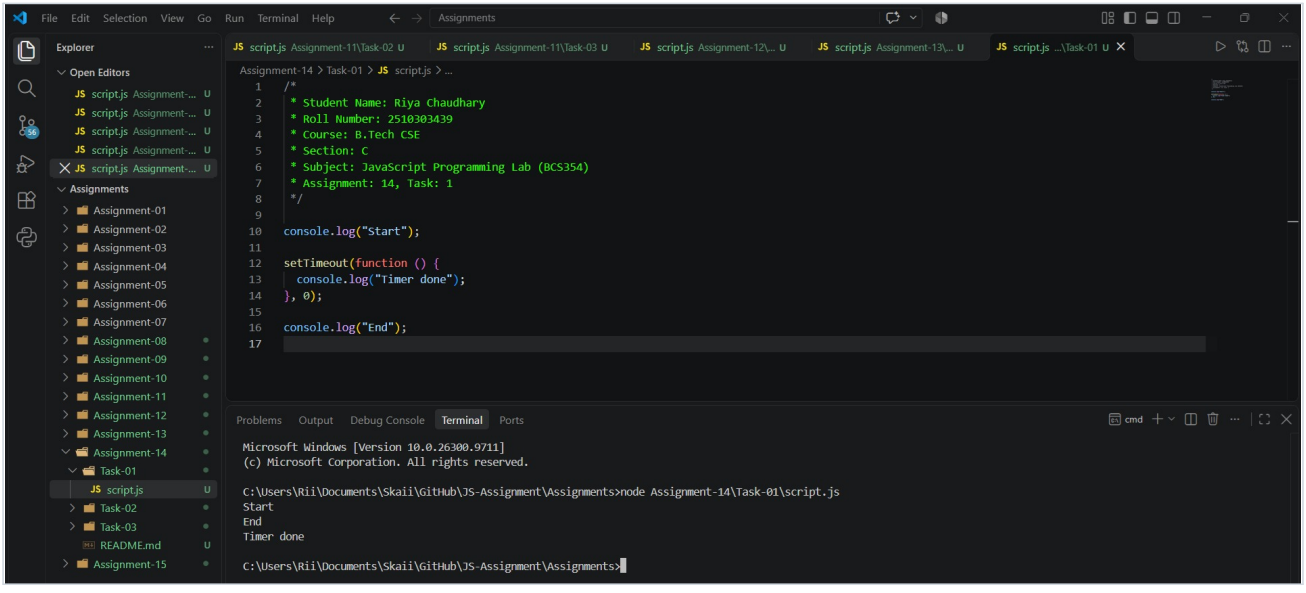
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 14, Task: 1
 */

console.log("Start");

setTimeout(function () {
  console.log("Timer done");
}, 0);

console.log("End");
```

Execution Output Screenshot:



Task 2: clearTimeout

Problem Statement: Schedule "Not shown" with setTimeout (1000 ms) and keep its id. Cancel it with clearTimeout. Print Cancelled.

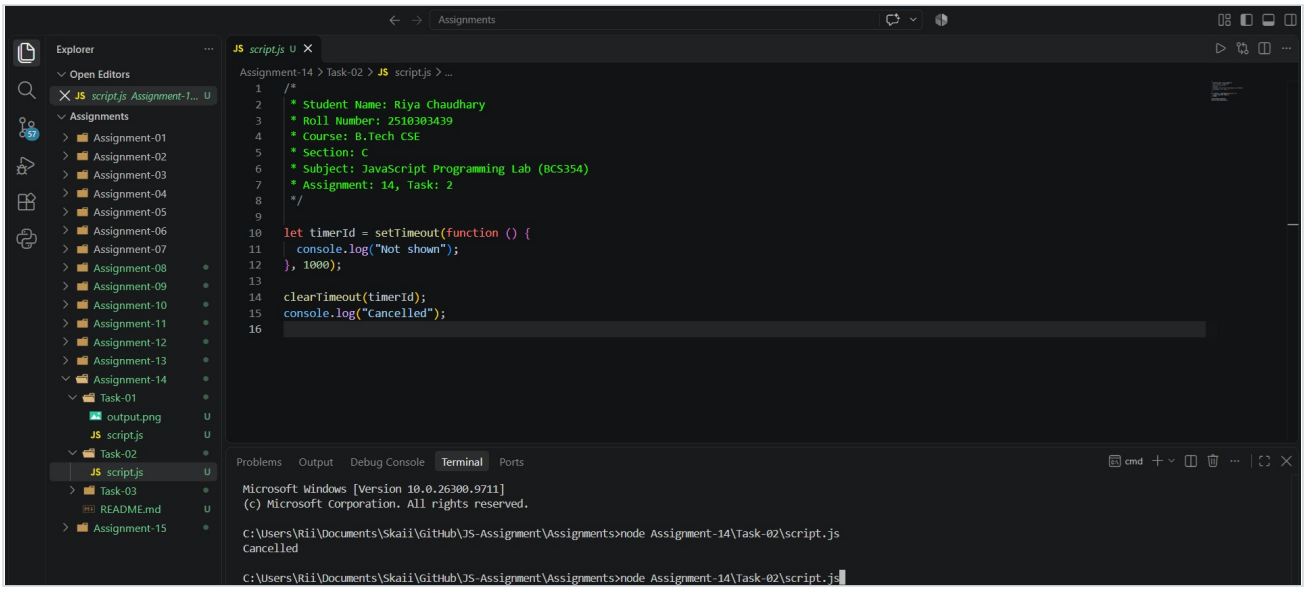
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 14, Task: 2
 */

let timerId = setTimeout(function () {
  console.log("Not shown");
}, 1000);

clearTimeout(timerId);
console.log("Cancelled");
```

Execution Output Screenshot:



Task 3: setInterval

Problem Statement: Set count = 0. Use setInterval every 200 ms to increase count and print Tick and the count. Stop it with clearInterval when count reaches 3.

Source Code (script.js):

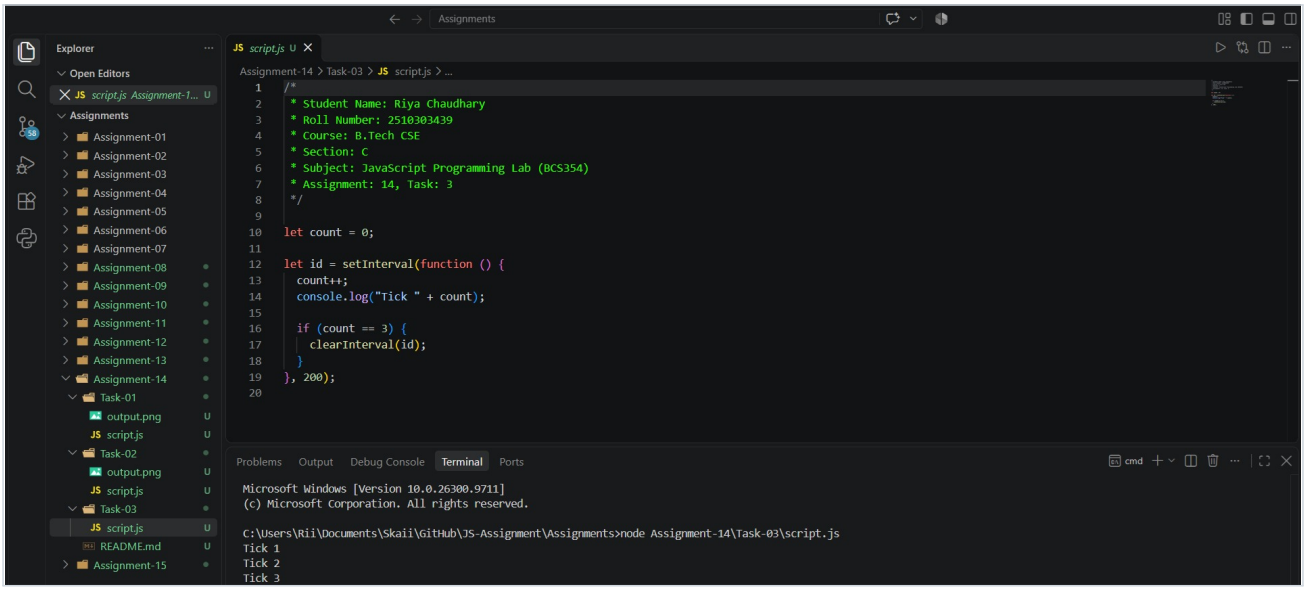
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 14, Task: 3
 */

let count = 0;

let id = setInterval(function () {
  count++;
  console.log("Tick " + count);

  if (count == 3) {
    clearInterval(id);
  }
}, 200);
```

Execution Output Screenshot:



Assignment 15: Promises and async/await with Local Data

Assignment Aim / Objective: To create a Promise that resolves or rejects, use async/await with try and catch, and load local data as a Promise.

Task 1: Promise resolves

Problem Statement: Create checkMarks(marks) returning a Promise. Resolve with marks for 0 to 100, otherwise reject with Error("Marks out of range"). Call checkMarks(80) and print the value.

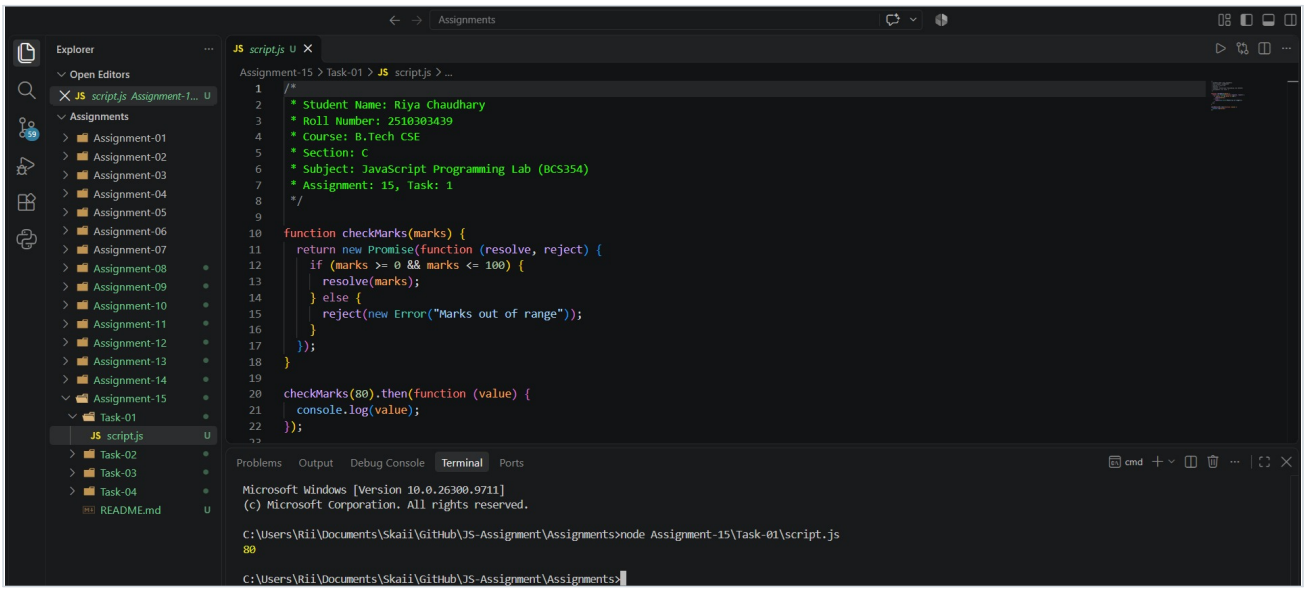
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 15, Task: 1
 */

function checkMarks(marks) {
  return new Promise(function (resolve, reject) {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

checkMarks(80).then(function (value) {
  console.log(value);
});
```

Execution Output Screenshot:



Task 2: Promise rejects

Problem Statement: Create checkMarks(marks) as before. Call checkMarks(120). Print the error message in catch.

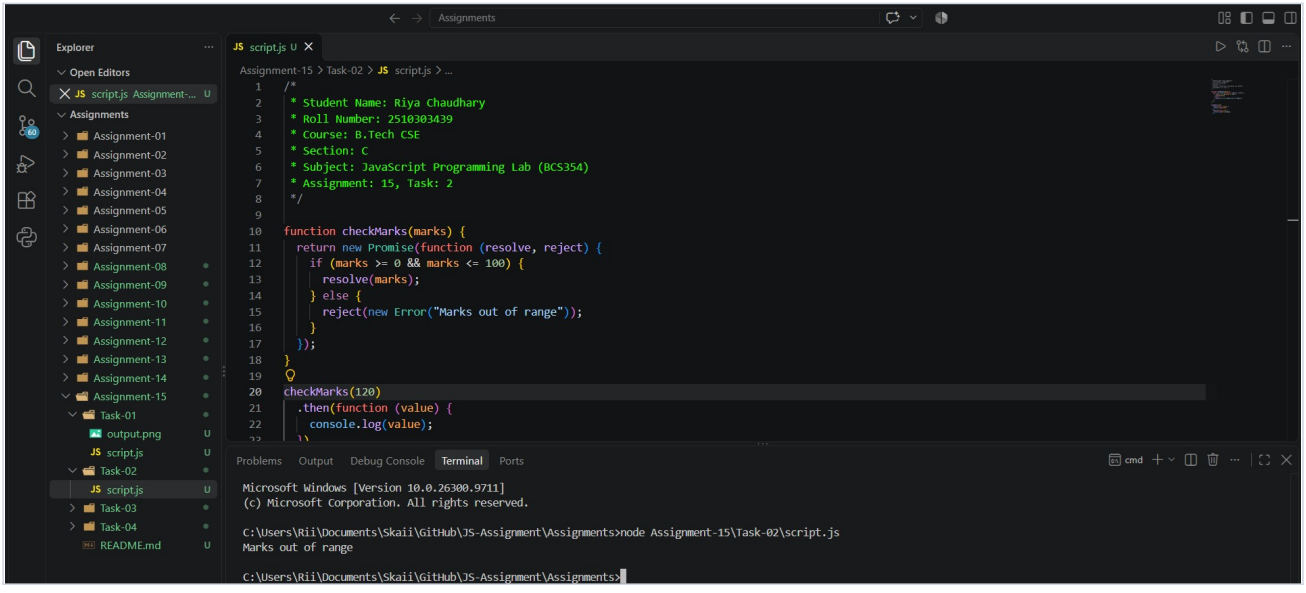
Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 15, Task: 2
 */

function checkMarks(marks) {
  return new Promise(function (resolve, reject) {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

checkMarks(120)
  .then(function (value) {
    console.log(value);
  })
  .catch(function (err) {
    console.log(err.message);
  });
```

Execution Output Screenshot:



Task 3: async/await with try and catch

Problem Statement: Create checkMarks(marks) as before. Create async function run(). In try, print await checkMarks(65), then await checkMarks(-5). In catch, print "Caught: " and the message. Call run().

Source Code (script.js):

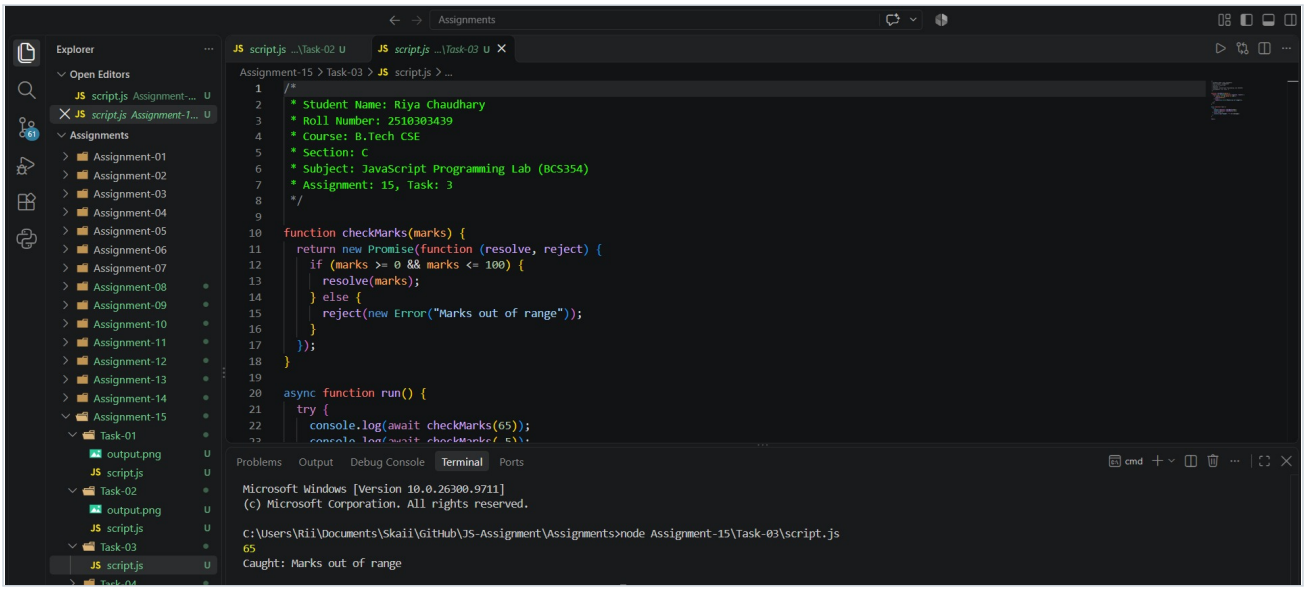
```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 15, Task: 3
 */

function checkMarks(marks) {
  return new Promise(function (resolve, reject) {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

async function run() {
  try {
    console.log(await checkMarks(65));
    console.log(await checkMarks(-5));
  } catch (err) {
    console.log("Caught: " + err.message);
  }
}

run();
```

Execution Output Screenshot:



Task 4: Load local users

Problem Statement: Create getUsers() returning a Promise that resolves after 100 ms with three users. Create an async function that awaits getUsers(). Print each user name. Call the function.

Source Code (script.js):

```
/*
 * Student Name: Riya Chaudhary
 * Roll Number: 2510303439
 * Course: B.Tech CSE
 * Section: C
 * Subject: JavaScript Programming Lab (BCS354)
 * Assignment: 15, Task: 4
 */

function getUsers() {
  return new Promise(function (resolve) {
    setTimeout(function () {
      resolve([
        { id: 1, name: "Asha" },
        { id: 2, name: "Rohan" },
        { id: 3, name: "Meera" }
      ]);
    }, 100);
  });
}

async function showUsers() {
  let users = await getUsers();

  users.forEach(function (u) {
    console.log(u.name);
  });
}

showUsers();
```

Execution Output Screenshot:

